

Urban Cart Retail Store

SQL Analytics Project

65 business questions answered in MySQL — from simple aggregates to joins and window functions — each with its query and its real output.

65

SQL QUERIES

6

SKILL LEVELS

2

LINKED TABLES

8.0

MYSQL

OVERVIEW

Contents at a glance

65

QUERIES

6

SKILL LEVELS

2

TABLES

1

DATABASE

THE SIX LEVELS

1

Aggregates & Calculations

Counting, summing, averaging, rounding and grouping order data to size up the business.

Q1-Q10

10 queries

PAGE

5

2

String Functions

Cleaning and reshaping customer text: names, emails, city codes and masked phone numbers.

Q11-Q18

8 queries

PAGE

10

3

Date & Time Functions

Delivery times, promised dates, customer tenure and monthly order trends using date arithmetic.

Q19-Q26

8 queries

PAGE

15

4

Conditional Logic with CASE

Segmenting customers and orders into income bands, order sizes and delivery speeds.

Q27-Q34

8 queries

PAGE

20

5

Joins & Multi-Table Analysis

INNER, LEFT, RIGHT and FULL joins that connect customers with orders and expose the gaps.

Q35-Q48

14 queries

PAGE

28

6

Window Functions & Analytics

Ranking, running totals, LAG/LEAD, quartile groups and share-of-total analysis.

Q49-Q65

17 queries

PAGE

37

HOW EACH QUERY IS PRESENTED

QUESTION → SQL QUERY → OUTPUT

PROJECT AIM

From raw retail data to answers

This project uses **SQL on MySQL** to turn the UrbanCart order and customer data into clear, business-ready answers. Sixty-five questions build up gradually: first summarising orders, then cleaning text and working with dates, then segmenting with conditions, joining the two tables, and finally ranking and comparing with window functions.

WORKFLOW

STEP 01

Set up

Create the database and load the customers and orders tables.

STEP 02

Query

Write one SQL statement for each business question.

STEP 03

Verify

Run each query and capture the result grid.

STEP 04

Present

Arrange question, query and output into this workbook.

SQL SKILLS COVERED

1

Aggregates & Calculations

COUNT · SUM · AVG · MIN/MAX · ROUND · CEILING/FLOOR · GROUP BY

2

String Functions

CONCAT · UPPER/LOWER · LENGTH · SUBSTRING · SUBSTRING_INDEX · REPEAT

3

Date & Time Functions

NOW · DATEDIFF · DATE_ADD/SUB · TIMESTAMPDIFF · MONTH/MONTHNAME

4

Conditional Logic with CASE

CASE WHEN · CTE (WITH) · GROUP BY on derived labels

5

Joins & Multi-Table Analysis

INNER / LEFT / RIGHT JOIN · UNION · HAVING · joined aggregates

6

Window Functions & Analytics

ROW_NUMBER · RANK · DENSE_RANK · NTILE · LAG · LEAD · SUM() OVER

TOOLS

MySQL 8.0 · MySQL Workbench · Database: urban_cart_retail_store

DATASET

Two tables, one common key

UrbanCart stores customers in one table and every purchase in another. Both share **customer_id**, which is the column used for every join question in this workbook.

customers

30 rows

Table: customers**Columns:**

customer_id	int
first_name	text
last_name	text
email	text
phone	bigint
city	text
state	text
signup_date	text
loyalty_tier	text
annual_income	int

orders

73 rows

Table: orders**Columns:**

order_id	int
customer_id	int
order_date	text
delivery_date	text
product_name	text
category	text
quantity	int
unit_price	int
discount_pct	int
payment_mode	text
ship_city	text

RELATIONSHIP

customers

customer_id (PK)

1:N

orders

customer_id (no FK)

The orders table deliberately has no foreign key. Five customers have never ordered, and a few orders belong to customer ids 101, 102 and 103 that do not exist in customers, so INNER, LEFT, RIGHT and FULL joins each return different results.

LEVEL 1

Aggregates & Calculations

Counting, summing, averaging, rounding and grouping order data to size up the business.

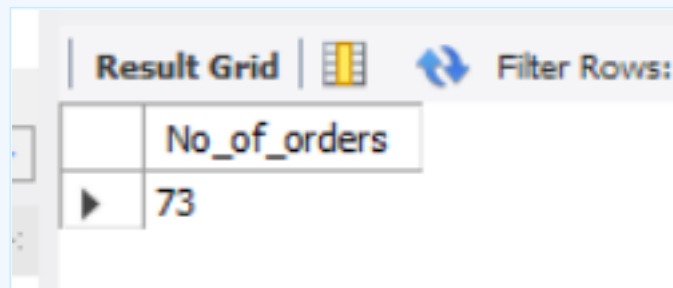
Q1 - Q10 · 10 queries

Q1**Find the total number of orders placed.**

SQL QUERY

```
select count(order_id) as No_of_orders from orders;
```

OUTPUT



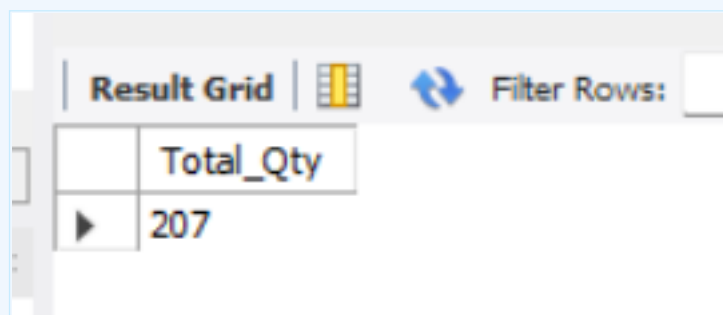
	No_of_orders
▶	73

Q2**Find the total quantity of items sold across all orders.**

SQL QUERY

```
select sum(quantity) as Total_Qty from orders;
```

OUTPUT



	Total_Qty
▶	207

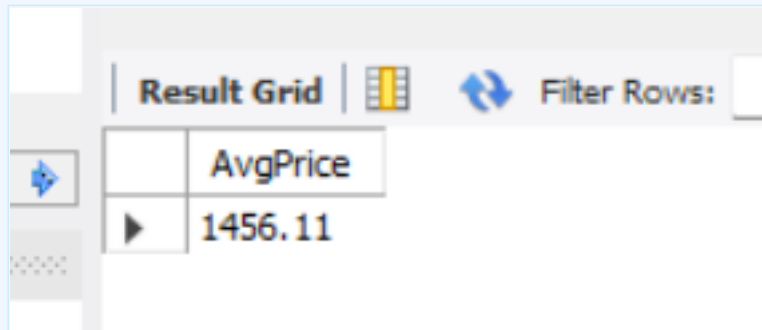
Q3

Find the average unit price of all products, rounded to 2 decimal places.

SQL QUERY

```
select round(avg(unit_price),2) as AvgPrice from orders;
```

OUTPUT



AvgPrice
1456.11

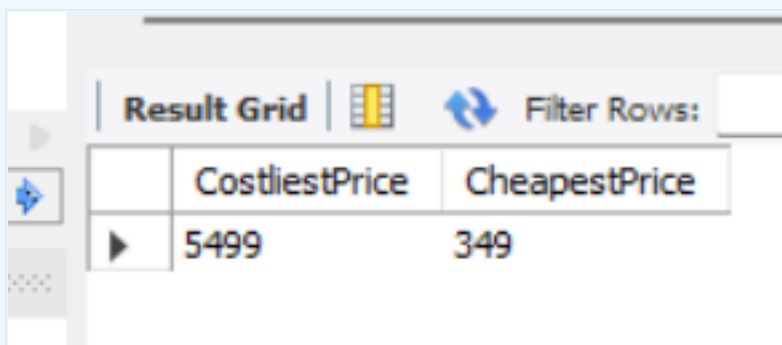
Q4

Show the cheapest and the costliest unit price in the orders table.

SQL QUERY

```
select max(unit_price) as CostliestPrice,min(Unit_price) as CheapestPrice from orders;
```

OUTPUT



CostliestPrice	CheapestPrice
5499	349

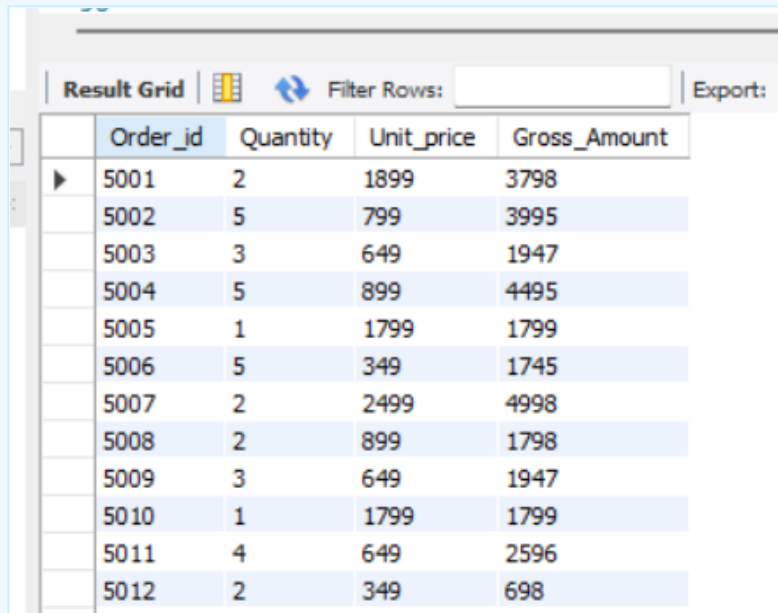
Q5

For every order, show the order id, quantity, unit price and the gross amount (quantity multiplied by unit price).

SQL QUERY

```
select Order_id,Quantity,Unit_price,round((unit_price)*quantity,2) as Gross_Amount from orders;
```

OUTPUT



	Order_id	Quantity	Unit_price	Gross_Amount
▶	5001	2	1899	3798
	5002	5	799	3995
	5003	3	649	1947
	5004	5	899	4495
	5005	1	1799	1799
	5006	5	349	1745
	5007	2	2499	4998
	5008	2	899	1798
	5009	3	649	1947
	5010	1	1799	1799
	5011	4	649	2596
	5012	2	349	698

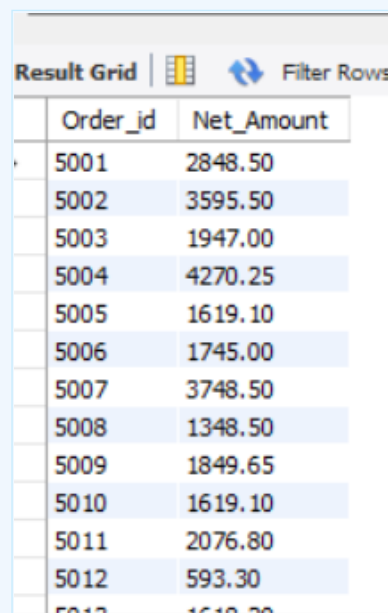
Q6

For every order, calculate the net amount after applying the discount percentage. Round it to 2 decimals.

SQL QUERY

```
select Order_id,round((unit_price-unit_price*(discount_pct/100))*quantity,2) as Net_Amount from orders;
```

OUTPUT



	Order_id	Net_Amount
▶	5001	2848.50
	5002	3595.50
	5003	1947.00
	5004	4270.25
	5005	1619.10
	5006	1745.00
	5007	3748.50
	5008	1348.50
	5009	1849.65
	5010	1619.10
	5011	2076.80
	5012	593.30

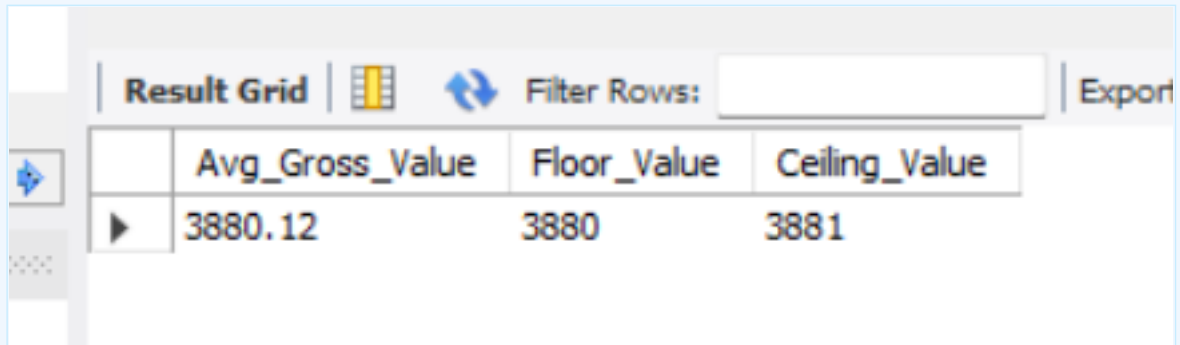
Q7

Show the average gross order value along with its CEILING and FLOOR values.

SQL QUERY

```
select round(avg(unit_price*quantity),2) as Avg_Gross_Value,  
       floor(avg(unit_price*quantity)) as Floor_Value,  
       ceiling(avg(unit_price*quantity)) as Ceiling_Value  
from orders;
```

OUTPUT



	Avg_Gross_Value	Floor_Value	Ceiling_Value
	3880.12	3880	3881

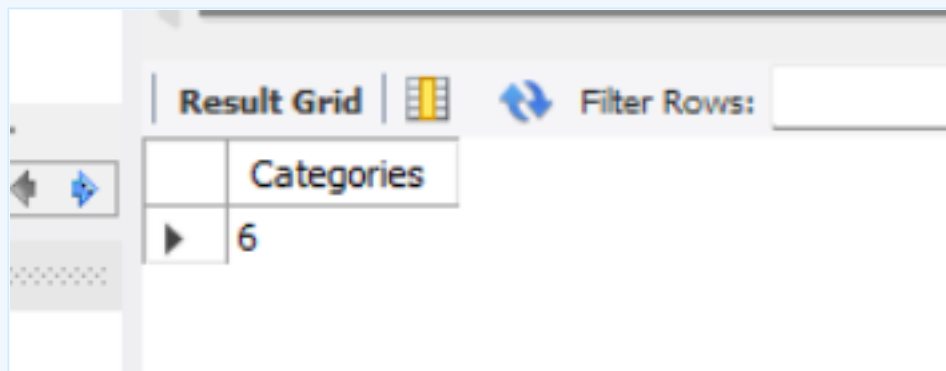
Q8

How many different product categories are sold?

SQL QUERY

```
select count(distinct category) as Categories from orders;
```

OUTPUT



Categories
6

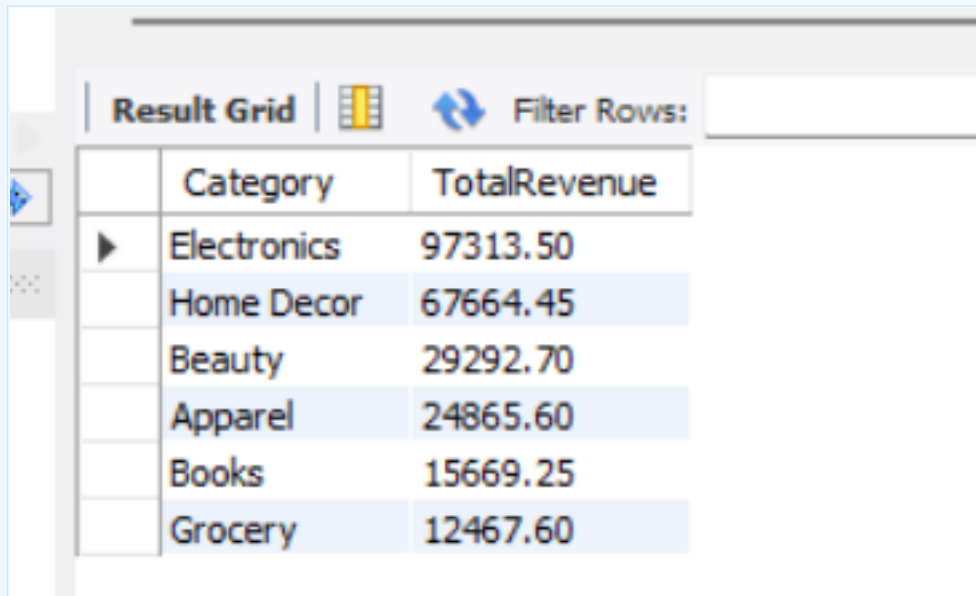
Q9

Show category wise total revenue after discount, highest revenue first.

SQL QUERY

```
select Category,round(sum((unit_price-unit_price*(discount_pct/100))*quantity),2) as TotalRevenue
from orders
group by category order by round(sum((unit_price-unit_price*(discount_pct/100))*quantity),2) desc ;
```

OUTPUT



	Category	TotalRevenue
▶	Electronics	97313.50
	Home Decor	67664.45
	Beauty	29292.70
	Apparel	24865.60
	Books	15669.25
	Grocery	12467.60

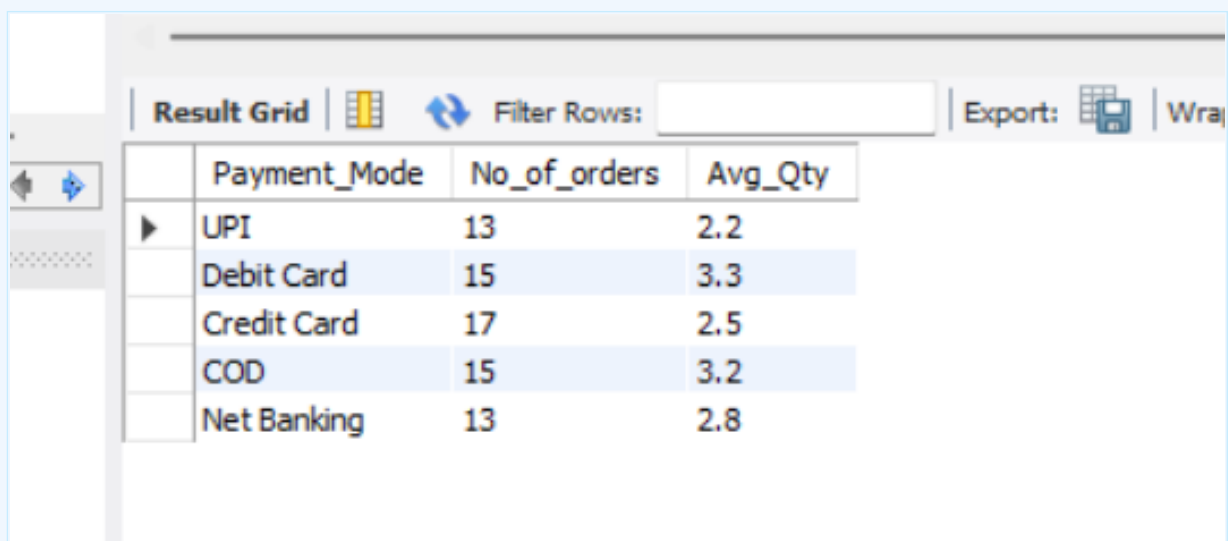
Q10

For each payment mode, show the number of orders and the average quantity rounded to 1 decimal place.

SQL QUERY

```
select Payment_Mode,count(order_id) as No_of_orders,round(avg(quantity),1) as Avg_Qty from
orders group by payment_mode;
```

OUTPUT



	Payment_Mode	No_of_orders	Avg_Qty
▶	UPI	13	2.2
	Debit Card	15	3.3
	Credit Card	17	2.5
	COD	15	3.2
	Net Banking	13	2.8

LEVEL 2

String Functions

Cleaning and reshaping customer text: names, emails, city codes and masked phone numbers.

Q11 - Q18 · 8 queries

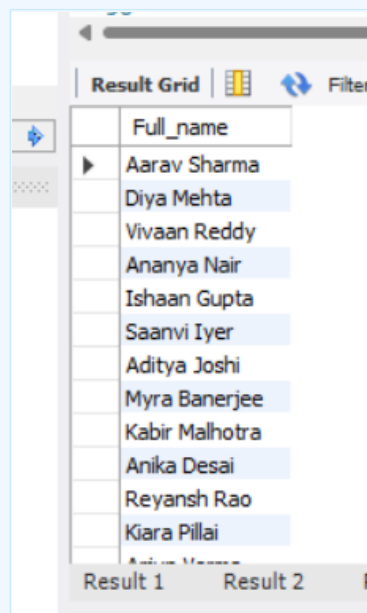
Q11

Display the full name of every customer in a single column.

SQL QUERY

```
select concat(first_name," ",Last_name) as Full_name from customers;
```

OUTPUT



Full_name
Aarav Sharma
Diya Mehta
Vivaan Reddy
Ananya Nair
Ishaan Gupta
Saanvi Iyer
Aditya Joshi
Myra Banerjee
Kabir Malhotra
Anika Desai
Reyansh Rao
Kiara Pillai
Anika Mehta

Q12 Display each customer's email in capital letters and city in small letters.

SQL QUERY

```
select upper(email) as EMAIL, lower(city) as city from customers;
```

OUTPUT

EMAIL	city
AARAV.SHARMA@GMAIL.COM	bengaluru
DIYA.MEHTA@YAHOO.COM	mumbai
VIVAAN.REDDY@OUTLOOK.COM	hyderabad
ANANYA.NAIR@REDIFFMAIL.COM	kochi
ISHAAN.GUPTA@GMAIL.COM	delhi
SAANVI.IYER@YAHOO.COM	chennai
ADITYA.JOSHI@OUTLOOK.COM	pune
MYRA.BANERJEE@REDIFFMAIL.COM	kolkata
KABIR.MALHOTRA@GMAIL.COM	delhi
ANIKA.DESAI@YAHOO.COM	ahmedabad
REYANSH.RAO@OUTLOOK.COM	bengaluru
KIARA.PILLAI@REDIFFMAIL.COM	chennai

Q13 Show the full name of each customer along with the number of characters in that name.

SQL QUERY

```
select concat(first_name, " ", last_name) as FullName,  
       char_length(first_name)+char_length(last_name) as Length  
from customers;
```

OUTPUT

FullName	Length
Aarav Sharma	11
Diya Mehta	9
Vivaan Reddy	11
Ananya Nair	10
Ishaan Gupta	11
Saanvi Iyer	10
Aditya Joshi	11
Myra Banerjee	12
Kabir Malhotra	13
Anika Desai	10
Reyansh Rao	10
Kiara Pillai	11

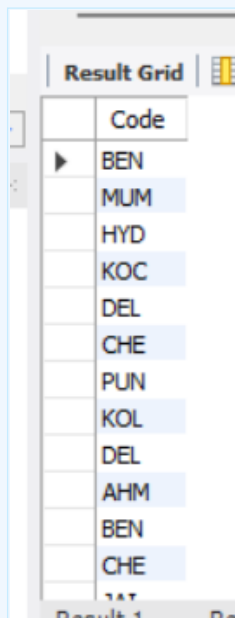
Q14

Create a 3 letter city code in capital letters from the city column (example: Bengaluru becomes BEN).

SQL QUERY

```
select upper(substring(city,1,3)) as Code from customers;
```

OUTPUT



Code
BEN
MUM
HYD
KOC
DEL
CHE
PUN
KOL
DEL
AHM
BEN
CHE

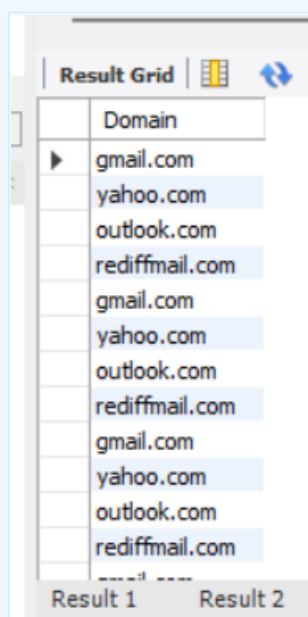
Q15

Extract only the email domain (the part after the @ symbol) for every customer.

SQL QUERY

```
select substring_index(email,"@",-1) as Domain from customers;
```

OUTPUT



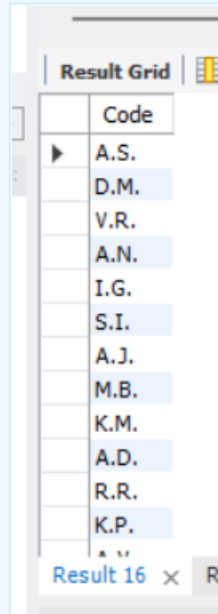
Domain
gmail.com
yahoo.com
outlook.com
rediffmail.com
gmail.com
yahoo.com
outlook.com
rediffmail.com
gmail.com
yahoo.com
outlook.com
rediffmail.com

Q16**Display the initials of every customer in the format A.S. for Aarav Sharma.**

SQL QUERY

```
select concat(substring(first_name,1,1),".",substring(last_name,1,1),".") as Code from customers;
```

OUTPUT



A screenshot of a database application's 'Result Grid' window. The window has a title bar with 'Result Grid' and a maximize button. It contains a table with one column labeled 'Code'. The table lists the initials of 16 customers: A.S., D.M., V.R., A.N., I.G., S.I., A.J., M.B., K.M., A.D., R.R., K.P., and then three more rows that are partially obscured. The window has a standard Windows-style title bar and a close button.

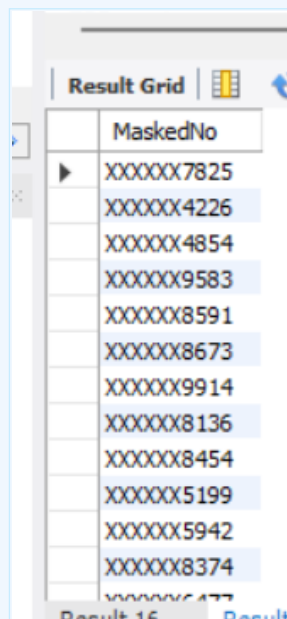
Code
A.S.
D.M.
V.R.
A.N.
I.G.
S.I.
A.J.
M.B.
K.M.
A.D.
R.R.
K.P.

Q17**Mask the phone number so that only the last 4 digits are visible, like XXXXXX1234.**

SQL QUERY

```
select concat(repeat("X",length(phone)-4), right(phone,4)) as MaskedNo from customers;
```

OUTPUT



A screenshot of a database application's 'Result Grid' window. The window has a title bar with 'Result Grid' and a maximize button. It contains a table with one column labeled 'MaskedNo'. The table lists 16 masked phone numbers, each consisting of six 'X' characters followed by the last four digits of the original phone number: XXXXXX7825, XXXXXX4226, XXXXXX4854, XXXXXX9583, XXXXXX8591, XXXXXX8673, XXXXXX9914, XXXXXX8136, XXXXXX8454, XXXXXX5199, XXXXXX5942, XXXXXX8374, and then three more rows that are partially obscured. The window has a standard Windows-style title bar and a close button.

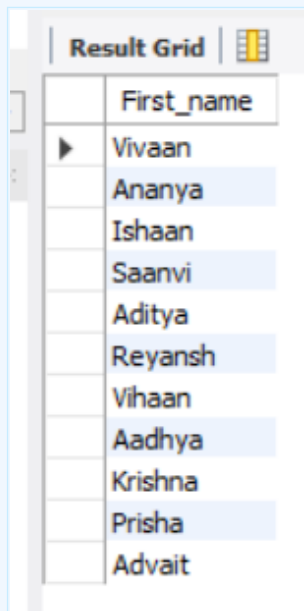
MaskedNo
XXXXXX7825
XXXXXX4226
XXXXXX4854
XXXXXX9583
XXXXXX8591
XXXXXX8673
XXXXXX9914
XXXXXX8136
XXXXXX8454
XXXXXX5199
XXXXXX5942
XXXXXX8374

Q18**List all customers whose first name has more than 5 characters.**

SQL QUERY

```
select First_name from customers where length(first_name)>5;
```

OUTPUT



The screenshot shows a 'Result Grid' window with a table containing 11 rows. The first row is the header 'First_name'. The subsequent rows list the first names of customers whose first name length is greater than 5 characters. The names are: Vivaan, Ananya, Ishaan, Saanvi, Aditya, Reyansh, Vihaan, Aadhya, Krishna, Prisha, and Advait.

First_name
Vivaan
Ananya
Ishaan
Saanvi
Aditya
Reyansh
Vihaan
Aadhya
Krishna
Prisha
Advait

LEVEL 3

Date & Time Functions

Delivery times, promised dates, customer tenure and monthly order trends using date arithmetic.

Q19 - Q26 · 8 queries

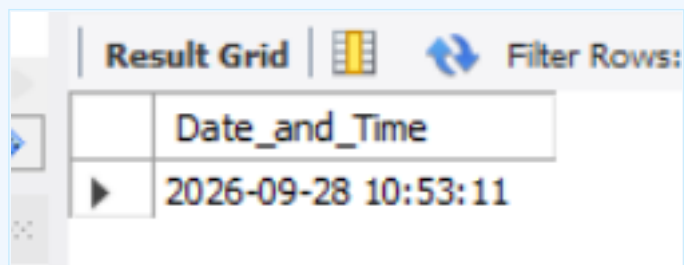
Q19

Display the current date and time of the database server.

SQL QUERY

```
select now() as Date_and_Time;
```

OUTPUT



	Date_and_Time
▶	2026-09-28 10:53:11

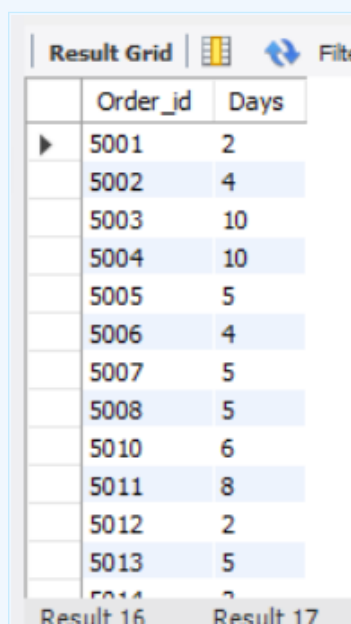
Q20

For every delivered order, show how many days the delivery took.

SQL QUERY

```
select Order_id ,datediff(delivery_date,Order_date) as Days from orders where delivery_date!="";
```

OUTPUT



	Order_id	Days
▶	5001	2
	5002	4
	5003	10
	5004	10
	5005	5
	5006	4
	5007	5
	5008	5
	5010	6
	5011	8
	5012	2
	5013	5
	5014	2

Result 16 Result 17

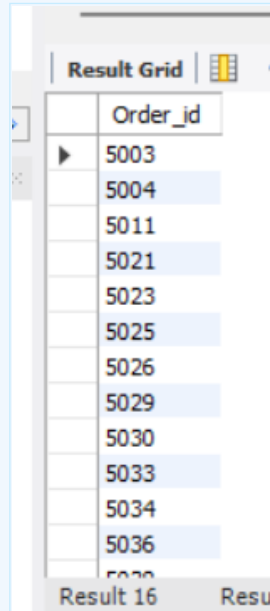
Q21

List the orders that took more than 7 days to reach the customer.

SQL QUERY

```
select Order_id from orders where datediff(delivery_date,order_date)>7;
```

OUTPUT



A screenshot of a database result grid titled 'Result Grid'. It displays a single column of order IDs. The visible values are 5003, 5004, 5011, 5021, 5023, 5025, 5026, 5029, 5030, 5033, 5034, and 5036. The grid has a scroll bar on the right, and the bottom of the grid shows 'Result 16' and 'Result 17'.

Order_id
5003
5004
5011
5021
5023
5025
5026
5029
5030
5033
5034
5036

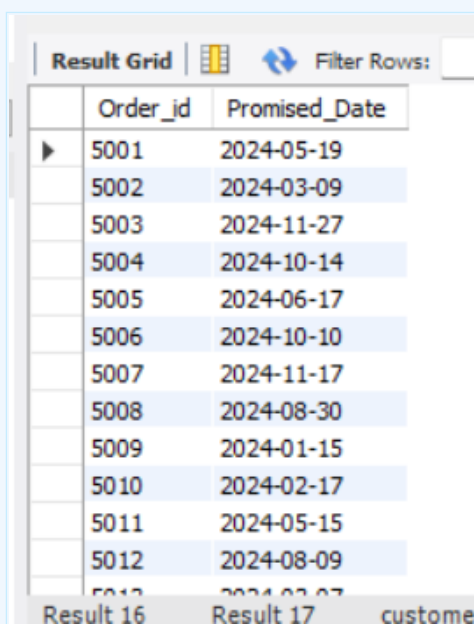
Q22

The company promises delivery within 7 days. Show the promised delivery date for every order.

SQL QUERY

```
select Order_id , date_add(order_date,interval 7 day) as Promised_Date from orders;
```

OUTPUT



A screenshot of a database result grid titled 'Result Grid'. It displays two columns: 'Order_id' and 'Promised_Date'. The data rows show order IDs from 5001 to 5012 with their corresponding promised delivery dates. The grid has a scroll bar on the right, and the bottom of the grid shows 'Result 16', 'Result 17', and 'Result 18'.

Order_id	Promised_Date
5001	2024-05-19
5002	2024-03-09
5003	2024-11-27
5004	2024-10-14
5005	2024-06-17
5006	2024-10-10
5007	2024-11-17
5008	2024-08-30
5009	2024-01-15
5010	2024-02-17
5011	2024-05-15
5012	2024-08-09

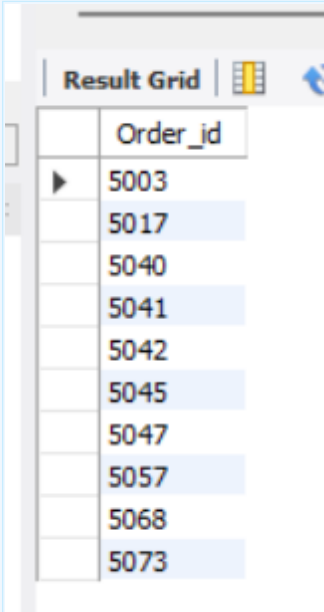
Q23

List all orders placed in the last 45 days of the year 2024 (that is, in the 45 days before 31 December 2024).

SQL QUERY

```
select Order_id from orders
where order_date between date_sub('2024-12-31', interval 44 day) and '2024-12-31';
```

OUTPUT



The screenshot shows a 'Result Grid' window with a table containing order IDs. The table has one column, 'Order_id', and ten rows of data. The order IDs are 5003, 5017, 5040, 5041, 5042, 5045, 5047, 5057, 5068, and 5073. The first row is highlighted with a blue background.

Order_id
5003
5017
5040
5041
5042
5045
5047
5057
5068
5073

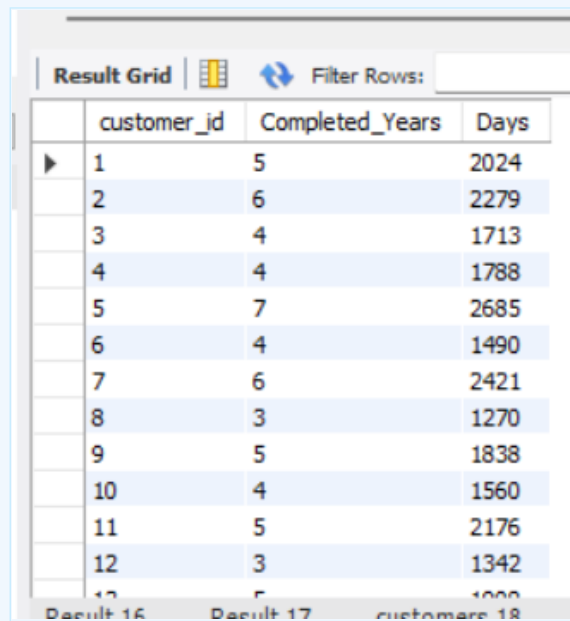
Q24

For every customer, show how many days they have been with the company and how many completed years that is.

SQL QUERY

```
select customer_id,  
       timestampdiff(year, signup_date, curdate()) as Completed_Years,  
       datediff(curdate(), signup_date) as Days  
from customers;
```

OUTPUT



The screenshot shows a database interface with a 'Result Grid' tab selected. The grid displays the results of the SQL query, showing columns for customer_id, Completed_Years, and Days. The data is presented in a table with alternating light blue and white rows. The first 12 rows are visible, showing customer IDs from 1 to 12. The bottom of the grid shows a scroll bar and a status bar with 'Result 16', 'Result 17', and 'customers 18'.

	customer_id	Completed_Years	Days
▶	1	5	2024
	2	6	2279
	3	4	1713
	4	4	1788
	5	7	2685
	6	4	1490
	7	6	2421
	8	3	1270
	9	5	1838
	10	4	1560
	11	5	2176
	12	3	1342
	13	5	1000

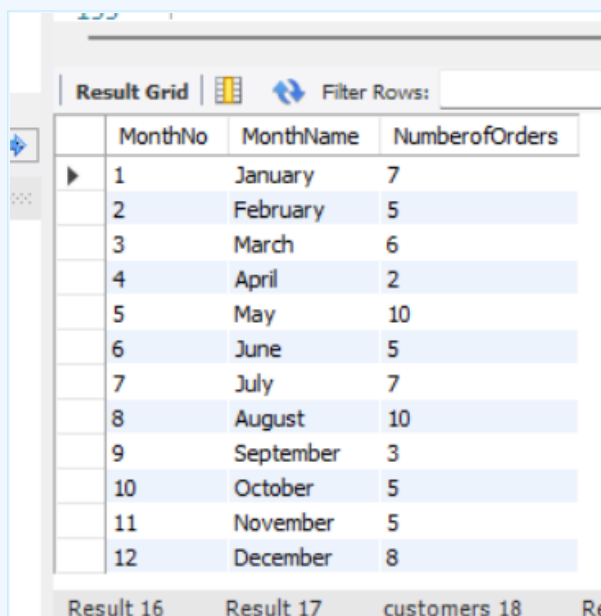
Q25

Show the number of orders placed in each month.

SQL QUERY

```
select month(order_date) as MonthNo, monthname(order_date) as MonthName, count(order_id) as  
NumberofOrders from orders group by month(order_date), monthname(order_date) order by  
month(order_date) asc;
```

OUTPUT



The screenshot shows a 'Result Grid' window with a table containing 12 rows of data. The columns are 'MonthNo', 'MonthName', and 'NumberofOrders'. The data is sorted by month number in ascending order. The table is displayed within a window that also shows 'Filter Rows' and other result sets like 'Result 16', 'Result 17', and 'customers 18'.

	MonthNo	MonthName	NumberofOrders
▶	1	January	7
	2	February	5
	3	March	6
	4	April	2
	5	May	10
	6	June	5
	7	July	7
	8	August	10
	9	September	3
	10	October	5
	11	November	5
	12	December	8

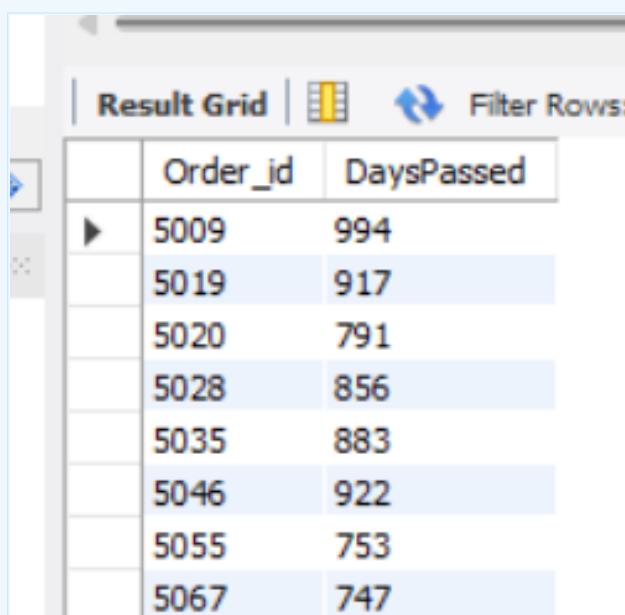
Q26

List all orders that are still not delivered and show how many days have passed since they were placed.

SQL QUERY

```
select Order_id ,datediff(curdate(),order_date) as DaysPassed from orders where delivery_date="";
```

OUTPUT



The screenshot shows a 'Result Grid' window with a table containing 8 rows of data. The columns are 'Order_id' and 'DaysPassed'. The data lists order IDs and the number of days since they were placed, for orders that have not yet been delivered. The table is displayed within a window that also shows 'Filter Rows' and other result sets like 'Result 16', 'Result 17', and 'customers 18'.

	Order_id	DaysPassed
▶	5009	994
	5019	917
	5020	791
	5028	856
	5035	883
	5046	922
	5055	753
	5067	747

LEVEL 4

Conditional Logic with CASE

Segmenting customers and orders into income bands, order sizes and delivery speeds.

Q27 - Q34 • 8 queries

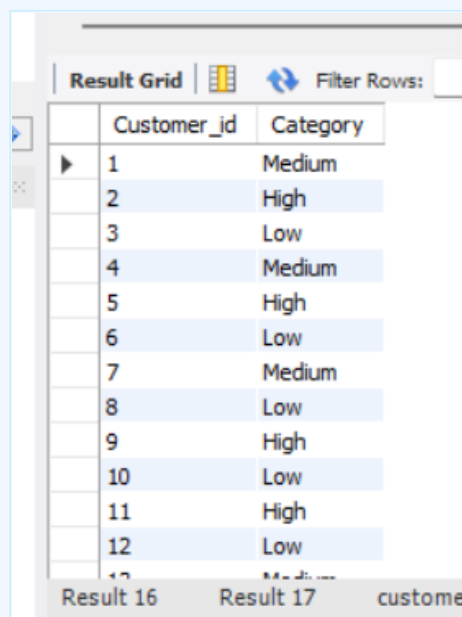
Q27

Label each customer's income as 'High', 'Medium' or 'Low' — High for 1,000,000 and above, Medium for 700,000 up to but not including 1,000,000, and Low for anything below 700,000.

SQL QUERY

```
select Customer_id,  
case  
when annual_income>=1000000 then "High"  
when annual_income>=700000 then "Medium"  
else "Low"  
end as Category  
from customers;
```

OUTPUT



The screenshot shows a 'Result Grid' window with a table containing two columns: 'Customer_id' and 'Category'. The table lists 12 customers with their corresponding income categories. The categories are: Medium, High, Low, Medium, High, Low, Medium, Low, High, Low, High, Low. The window also includes a 'Filter Rows' button and a status bar at the bottom showing 'Result 16', 'Result 17', and 'custome'.

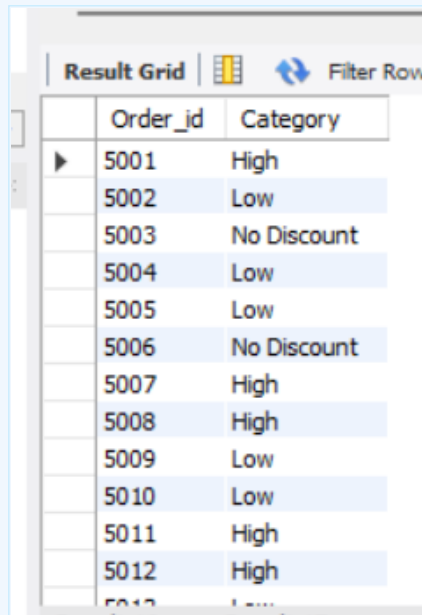
Customer_id	Category
1	Medium
2	High
3	Low
4	Medium
5	High
6	Low
7	Medium
8	Low
9	High
10	Low
11	High
12	Low

Q28

For every order, show whether the discount given was 'No Discount', 'Low' (up to 10 percent), or 'High' (above 10 percent).

SQL QUERY

```
select Order_id,  
case  
when discount_pct="" then "No Discount"  
when discount_pct<=10 then "Low"  
else "High"  
end as Category  
from orders;
```

OUTPUT

Order_id	Category
5001	High
5002	Low
5003	No Discount
5004	Low
5005	Low
5006	No Discount
5007	High
5008	High
5009	Low
5010	Low
5011	High
5012	High
5013	Low

Q29

For every order, classify the order size as 'Small' (gross amount below 2000), 'Medium' (2000 up to 8000) or 'Large' (8000 and above).

SQL QUERY

```
select Order_id,  
case  
when unit_price*quantity>=8000 then "Large"  
when unit_price*quantity>=2000 then "Medium"  
else "Small"  
end as Category  
from orders;
```

OUTPUT

Order_id	Category
5001	Medium
5002	Medium
5003	Small
5004	Medium
5005	Small
5006	Small
5007	Medium
5008	Small
5009	Small
5010	Small
5011	Medium
5012	Small

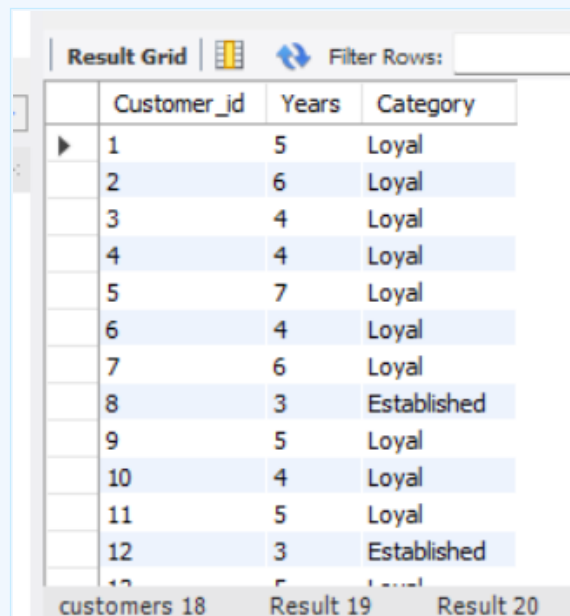
Q30

Show each customer's tenure with the company in years, and next to it a tag: 'New' for less than 2 years, 'Established' for 2 up to 4 years, and 'Loyal' for 4 years and above.

SQL QUERY

```
select Customer_id,timestampdiff(year,signup_date,curdate()) as Years,
case
when timestampdiff(year,signup_date,curdate())>=4 then "Loyal"
when timestampdiff(year,signup_date,curdate())>=2 then "Established"
else "New" end as Category
from customers;
```

OUTPUT



The screenshot shows a 'Result Grid' window with a table containing customer data. The table has three columns: 'Customer_id', 'Years', and 'Category'. The data is as follows:

Customer_id	Years	Category
1	5	Loyal
2	6	Loyal
3	4	Loyal
4	4	Loyal
5	7	Loyal
6	4	Loyal
7	6	Loyal
8	3	Established
9	5	Loyal
10	4	Loyal
11	5	Loyal
12	3	Established

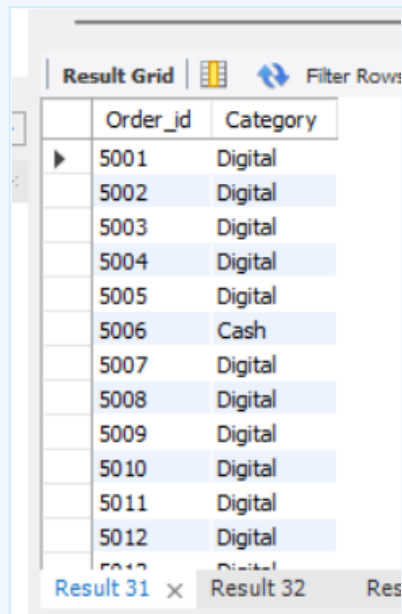
At the bottom of the window, there are tabs labeled 'customers 18', 'Result 19', and 'Result 20'.

Q31

For every order, show a simple payment type: 'Cash' for cash on delivery, and 'Digital' for every other payment mode.

SQL QUERY

```
select Order_id ,  
case  
when payment_mode="cod" then "Cash"  
else "Digital" end  
as Category from orders;
```

OUTPUT

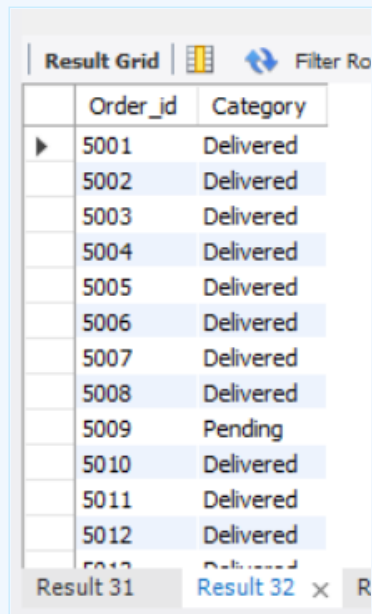
Order_id	Category
5001	Digital
5002	Digital
5003	Digital
5004	Digital
5005	Digital
5006	Cash
5007	Digital
5008	Digital
5009	Digital
5010	Digital
5011	Digital
5012	Digital
5013	Digital

Q32

For every order, show a delivery status: 'Delivered' if a delivery date exists, and 'Pending' if it does not.

SQL QUERY

```
select Order_id ,  
case  
when delivery_date!="" then "Delivered"  
else "Pending" end  
as Category from orders;
```

OUTPUT

The screenshot shows a 'Result Grid' window with a table containing two columns: 'Order_id' and 'Category'. The table lists 13 orders. Orders 5001 through 5008 and 5010 through 5012 are marked as 'Delivered'. Order 5009 is marked as 'Pending'. The window also shows a 'Filter Row' icon and a tab labeled 'Result 32'.

Order_id	Category
5001	Delivered
5002	Delivered
5003	Delivered
5004	Delivered
5005	Delivered
5006	Delivered
5007	Delivered
5008	Delivered
5009	Pending
5010	Delivered
5011	Delivered
5012	Delivered
5013	Delivered

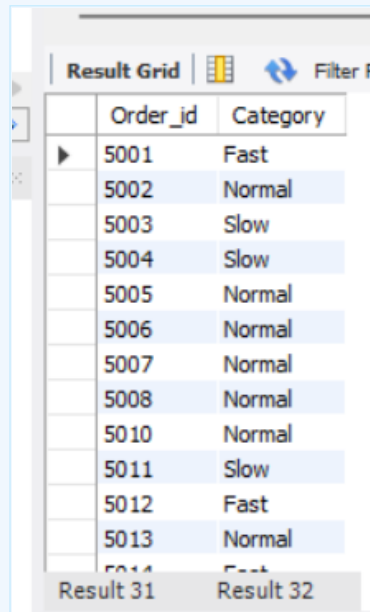
Q33

For delivered orders, tag the delivery speed as 'Fast' (3 days or less), 'Normal' (4 to 7 days) or 'Slow' (more than 7 days).

SQL QUERY

```
select Order_id ,  
case  
when datediff(delivery_date,order_date)>7 then "Slow"  
when datediff(delivery_date,order_date)>=4 then "Normal"  
else "Fast" end  
as Category from orders  
where delivery_date !="";
```

OUTPUT



Order_id	Category
5001	Fast
5002	Normal
5003	Slow
5004	Slow
5005	Normal
5006	Normal
5007	Normal
5008	Normal
5010	Normal
5011	Slow
5012	Fast
5013	Normal
5014	Fast

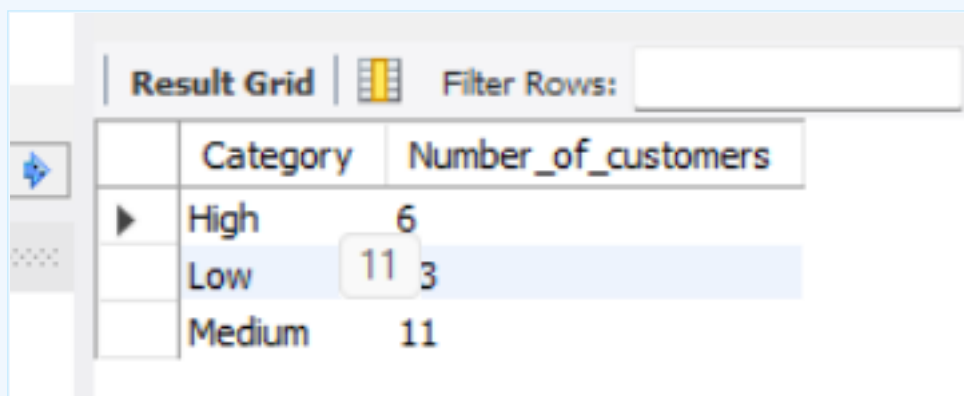
Q34

Count how many customers fall into each income band described earlier (High, Medium, Low) in a single result.

SQL QUERY

```
with cte as(  
  select Customer_id,  
         case  
           when annual_income>=1000000 then "High"  
           when annual_income>=700000 then "Medium"  
           else "Low"  
         end as Category  
  from customers  
)  
select Category, count(customer_id) as Number_of_customers from cte group by Category order by  
        Category asc;
```

OUTPUT



The screenshot shows a database interface with a 'Result Grid' tab. A table with two columns, 'Category' and 'Number_of_customers', is displayed. The table contains three rows: 'High' with 6 customers, 'Low' with 11 customers, and 'Medium' with 11 customers. The 'Low' row is highlighted. A 'Filter Rows' input field is visible at the top right of the grid.

	Category	Number_of_customers
▶	High	6
⋮	Low	11
	Medium	11

LEVEL 5

Joins & Multi-Table Analysis

INNER, LEFT, RIGHT and FULL joins that connect customers with orders and expose the gaps.

Q35 - Q48 · 14 queries

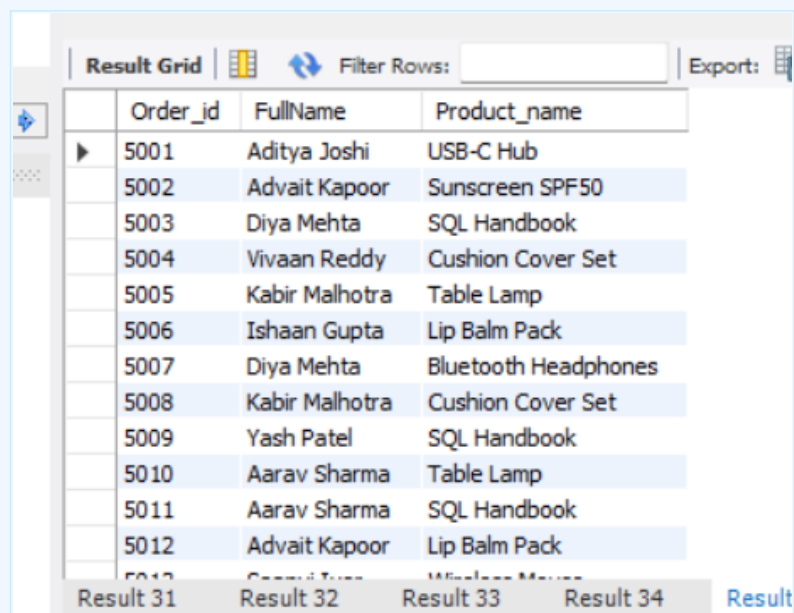
Q35

Show the order id, customer full name and product name for every order that has a matching customer record.

SQL QUERY

```
select o.Order_id,concat(c.first_name," ",c.last_name) as FullName, o.Product_name from orders as o
inner join customers as c
on c.customer_id= o.customer_id;
```

OUTPUT



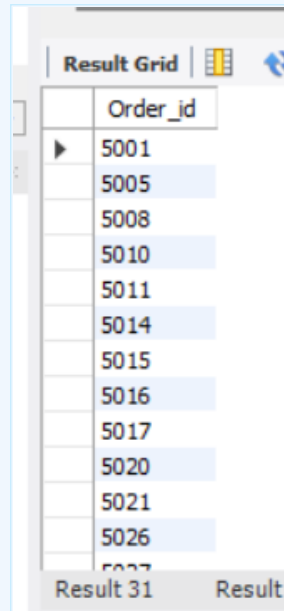
	Order_id	FullName	Product_name
▶	5001	Aditya Joshi	USB-C Hub
	5002	Advait Kapoor	Sunscreen SPF50
	5003	Diya Mehta	SQL Handbook
	5004	Vivaan Reddy	Cushion Cover Set
	5005	Kabir Malhotra	Table Lamp
	5006	Ishaan Gupta	Lip Balm Pack
	5007	Diya Mehta	Bluetooth Headphones
	5008	Kabir Malhotra	Cushion Cover Set
	5009	Yash Patel	SQL Handbook
	5010	Aarav Sharma	Table Lamp
	5011	Aarav Sharma	SQL Handbook
	5012	Advait Kapoor	Lip Balm Pack

Q36 List all orders placed by Gold tier customers only.

SQL QUERY

```
select o.Order_id from customers as c inner join orders as o on c.customer_id=o.customer_id
where c.loyalty_tier="Gold";
```

OUTPUT



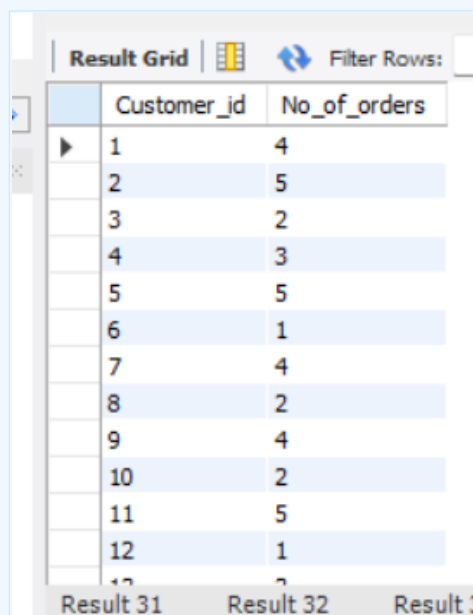
Order_id
5001
5005
5008
5010
5011
5014
5015
5016
5017
5020
5021
5026
5027

Q37 Show every customer along with the number of orders they have placed. Customers with zero orders must also appear.

SQL QUERY

```
select c.Customer_id, count(order_id) as No_of_orders from customers as c left join orders as o
on c.customer_id= o.customer_id group by Customer_id;
```

OUTPUT



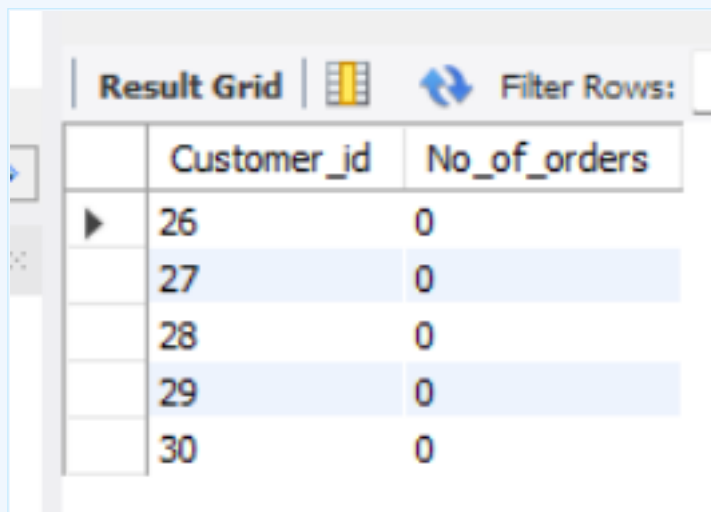
Customer_id	No_of_orders
1	4
2	5
3	2
4	3
5	5
6	1
7	4
8	2
9	4
10	2
11	5
12	1

Q38**List the customers who have never placed a single order.**

SQL QUERY

```
select c.Customer_id, count(order_id) as No_of_orders from customers as c left join orders as o
on c.customer_id= o.customer_id group by Customer_id having count(order_id)=0;
```

OUTPUT



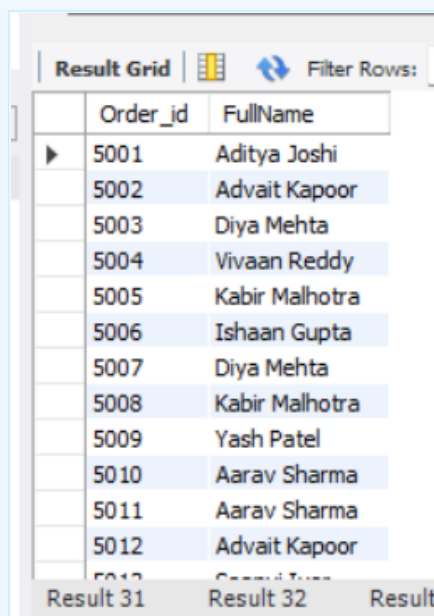
	Customer_id	No_of_orders
▶	26	0
	27	0
	28	0
	29	0
	30	0

Q39**Show all orders along with the customer name. Orders must appear even if the customer record is missing.**

SQL QUERY

```
select o.Order_id, concat(c.first_name, " ", c.last_name) as FullName from customers as c right join
orders as o
on c.customer_id= o.customer_id ;
```

OUTPUT



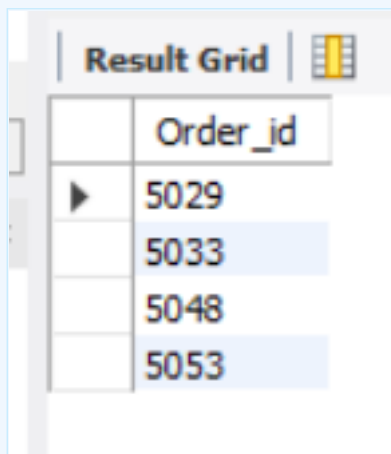
	Order_id	FullName
▶	5001	Aditya Joshi
	5002	Advait Kapoor
	5003	Diya Mehta
	5004	Vivaan Reddy
	5005	Kabir Malhotra
	5006	Ishaan Gupta
	5007	Diya Mehta
	5008	Kabir Malhotra
	5009	Yash Patel
	5010	Aarav Sharma
	5011	Aarav Sharma
	5012	Advait Kapoor

Q40**Find the orders whose customer record is missing from the customers table.**

SQL QUERY

```
select o.Order_id from orders as o left join customers as c
on c.customer_id=o.customer_id where c.Customer_id is NULL;
```

OUTPUT



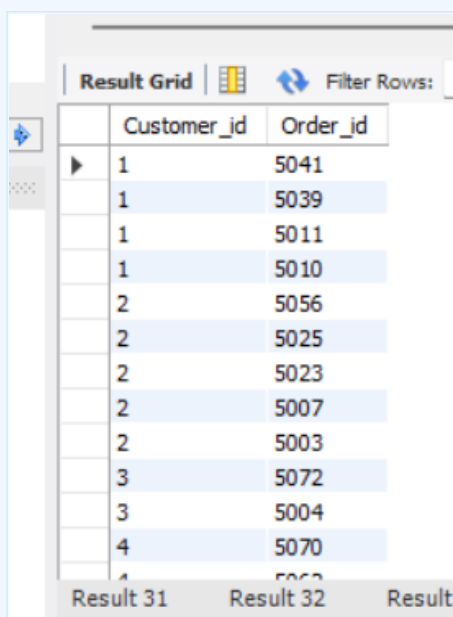
	Order_id
▶	5029
	5033
	5048
	5053

Q41**Combine every customer and every order into a single result, so that customers who never ordered and orders with no matching customer both still appear.**

SQL QUERY

```
select c.Customer_id , o.Order_id from customers as c left join orders as o
on c.customer_id=o.customer_id
union
select c.customer_id , o.order_id from customers as c right join orders as o
on c.customer_id=o.customer_id;
```

OUTPUT



	Customer_id	Order_id
▶	1	5041
	1	5039
	1	5011
	1	5010
	2	5056
	2	5025
	2	5023
	2	5007
	2	5003
	3	5072
	3	5004
	4	5070
	4	5062

Result 31 Result 32 Result

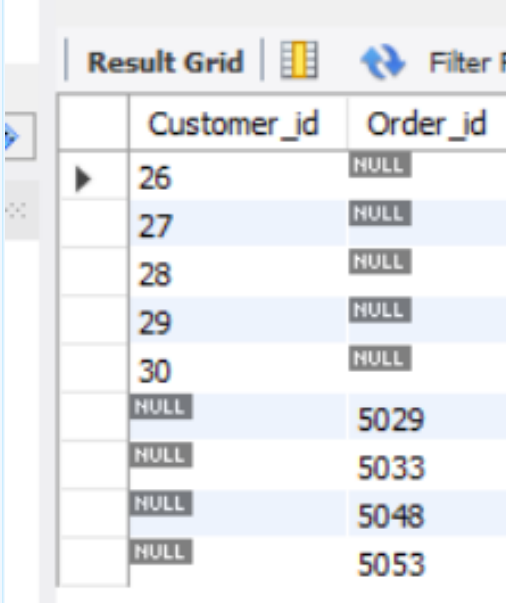
Q42

Show only the unmatched rows from both sides, that is, customers with no orders and orders with no customer.

SQL QUERY

```
select c.Customer_id , o.Order_id from customers as c left join orders as o
on c.customer_id=o.customer_id where o.order_id is null
union
select c.customer_id , o.order_id from customers as c right join orders as o
on c.customer_id=o.customer_id where c.customer_id is null;
```

OUTPUT



	Customer_id	Order_id
▶	26	NULL
	27	NULL
	28	NULL
	29	NULL
	30	NULL
	NULL	5029
	NULL	5033
	NULL	5048
	NULL	5053

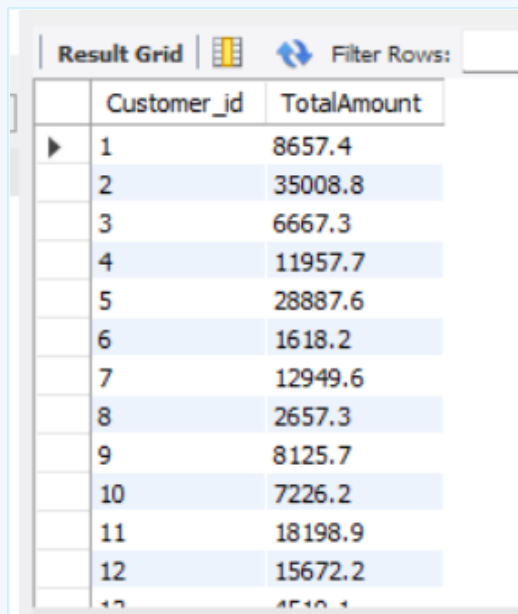
Q43

Find the total amount spent by each customer after discount. Show only customers who have placed at least one order.

SQL QUERY

```
select c.Customer_id,round(sum((o.unit_price-o.unit_price*(o.discount_pct/100))*o.quantity),1) as  
    TotalAmount from  
customers as c inner join orders as o on c.customer_id=o.customer_id group by c.Customer_id  
having count(order_id)>=1 order by c.Customer_id;
```

OUTPUT



The screenshot shows a 'Result Grid' window with a 'Filter Rows' button. The grid contains two columns: 'Customer_id' and 'TotalAmount'. The data is sorted by 'Customer_id' in ascending order. The first 12 rows are visible, showing customer IDs from 1 to 12 and their corresponding total amounts.

Customer_id	TotalAmount
1	8657.4
2	35008.8
3	6667.3
4	11957.7
5	28887.6
6	1618.2
7	12949.6
8	2657.3
9	8125.7
10	7226.2
11	18198.9
12	15672.2

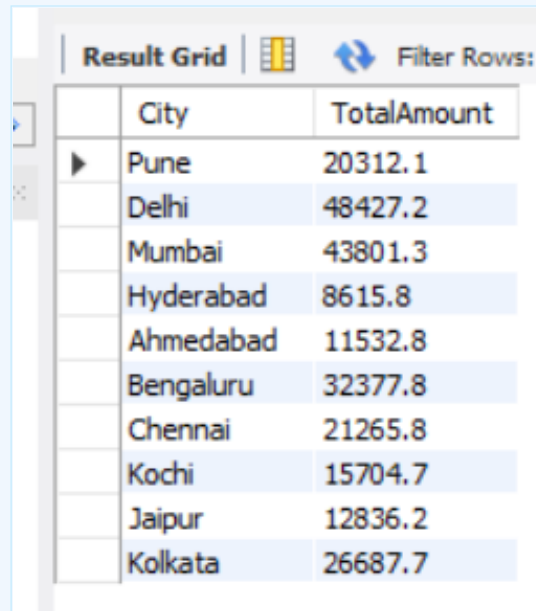
Q44

Find city wise revenue based on the customer's home city.

SQL QUERY

```
select c.City, round(sum((o.unit_price-o.unit_price*(o.discount_pct/100))*o.quantity),1) as  
TotalAmount from  
customers as c inner join orders as o on c.customer_id=o.customer_id group by c.city;
```

OUTPUT



The screenshot shows a 'Result Grid' with two columns: 'City' and 'TotalAmount'. The data is as follows:

City	TotalAmount
Pune	20312.1
Delhi	48427.2
Mumbai	43801.3
Hyderabad	8615.8
Ahmedabad	11532.8
Bengaluru	32377.8
Chennai	21265.8
Kochi	15704.7
Jaipur	12836.2
Kolkata	26687.7

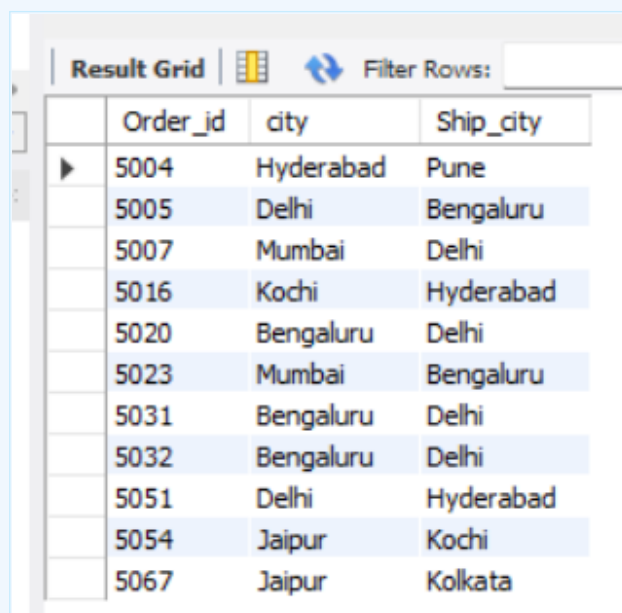
Q45

List the orders that were shipped to a city different from the customer's home city.

SQL QUERY

```
select o.Order_id, c.city,o.Ship_city from orders as o inner join customers as c on c.customer_id=  
o.customer_id where c.city!=o.Ship_city;
```

OUTPUT



The screenshot shows a 'Result Grid' with three columns: 'Order_id', 'city', and 'Ship_city'. The data is as follows:

Order_id	city	Ship_city
5004	Hyderabad	Pune
5005	Delhi	Bengaluru
5007	Mumbai	Delhi
5016	Kochi	Hyderabad
5020	Bengaluru	Delhi
5023	Mumbai	Bengaluru
5031	Bengaluru	Delhi
5032	Bengaluru	Delhi
5051	Delhi	Hyderabad
5054	Jaipur	Kochi
5067	Jaipur	Kolkata

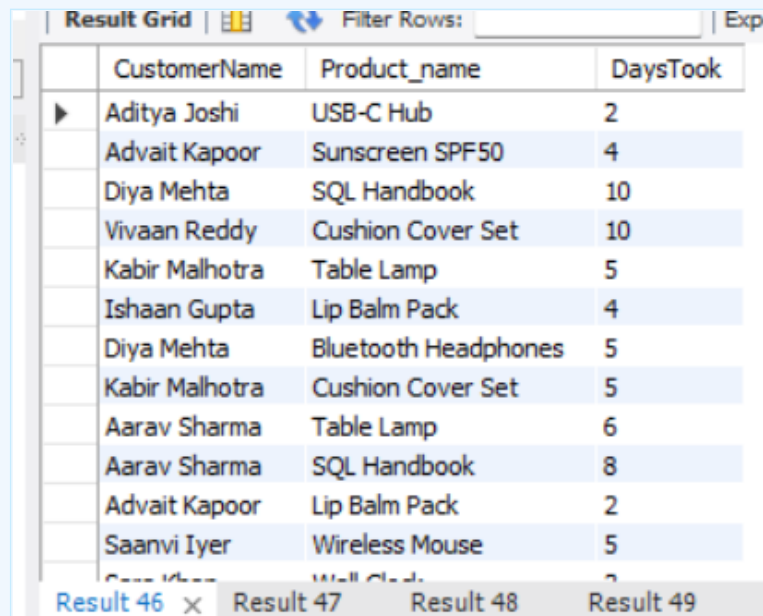
Q46

Show the customer name, product name and the number of days the delivery took, for delivered orders only.

SQL QUERY

```
select concat(c.first_name," ",c.last_name) as CustomerName, o.Product_name,
       datediff(o.delivery_date,o.order_date) as DaysTook
from customers as c inner join orders as o on c.customer_id=o.customer_id
where o.delivery_date!="";
```

OUTPUT



	CustomerName	Product_name	DaysTook
▶	Aditya Joshi	USB-C Hub	2
	Advait Kapoor	Sunscreen SPF50	4
	Diya Mehta	SQL Handbook	10
	Vivaan Reddy	Cushion Cover Set	10
	Kabir Malhotra	Table Lamp	5
	Ishaan Gupta	Lip Balm Pack	4
	Diya Mehta	Bluetooth Headphones	5
	Kabir Malhotra	Cushion Cover Set	5
	Aarav Sharma	Table Lamp	6
	Aarav Sharma	SQL Handbook	8
	Advait Kapoor	Lip Balm Pack	2
	Saanvi Iyer	Wireless Mouse	5
	Saanvi Iyer	Wall Clock	2

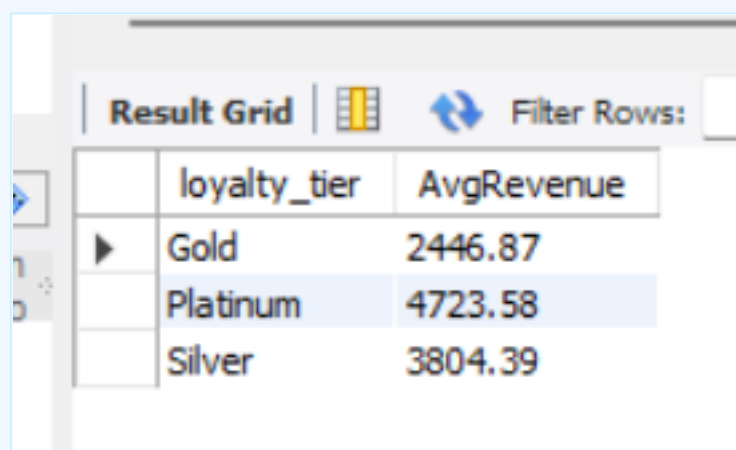
Q47

Find the average order value for each loyalty tier, rounded to 2 decimals.

SQL QUERY

```
select c.loyalty_tier,round(avg((o.unit_price-o.unit_price*(o.discount_pct/100))*o.quantity),2) as
       AvgRevenue
from customers as c inner join orders as o on c.customer_id=o.customer_id group by c.loyalty_tier;
```

OUTPUT



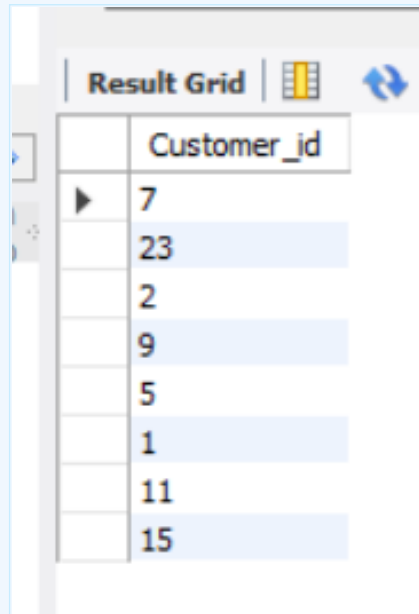
	loyalty_tier	AvgRevenue
▶	Gold	2446.87
	Platinum	4723.58
	Silver	3804.39

Q48**List the customers who have placed more than 3 orders.**

SQL QUERY

```
select c.Customer_id from customers as c inner join
orders as o on c.customer_id=o.customer_id group by c.customer_id having count(o.order_id)>3 ;
```

OUTPUT



Customer_id
7
23
2
9
5
1
11
15

LEVEL 6

Window Functions & Analytics

Ranking, running totals, LAG/LEAD, quartile groups and share-of-total analysis.

Q49 - Q65 · 17 queries

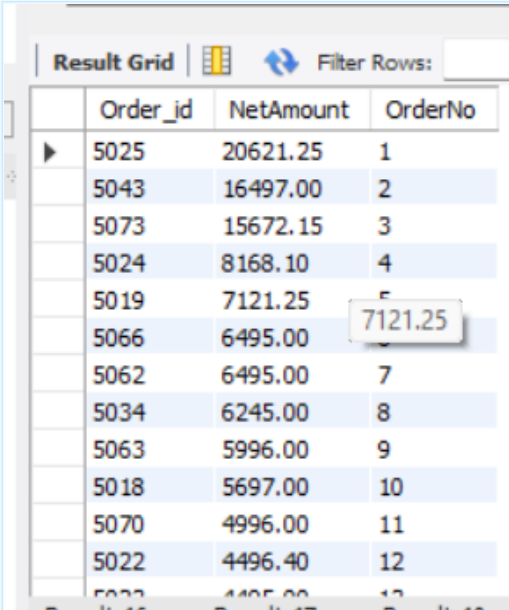
Q49

Number all orders from the highest net amount to the lowest.

SQL QUERY

```
select Order_id,  
       round((unit_price-unit_price*(discount_pct/100))*quantity,2) as NetAmount,  
       row_number() over (order by (unit_price-unit_price*(discount_pct/100))*quantity desc) as  
       OrderNo  
from orders;
```

OUTPUT



Order_id	NetAmount	OrderNo
5025	20621.25	1
5043	16497.00	2
5073	15672.15	3
5024	8168.10	4
5019	7121.25	5
5066	6495.00	6
5062	6495.00	7
5034	6245.00	8
5063	5996.00	9
5018	5697.00	10
5070	4996.00	11
5022	4496.40	12
5022	4496.40	13

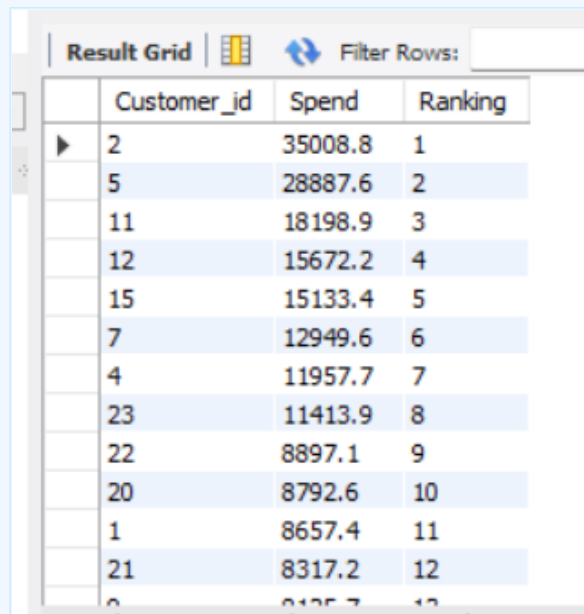
Q50

Rank the customers by their total spend, highest spender getting rank 1.

SQL QUERY

```
select c.Customer_id, round(sum((o.unit_price-o.unit_price*(o.discount_pct/100))*o.quantity),1) as  
Spend ,  
dense_rank() over (order by  
(round(sum((o.unit_price-unit_price*(o.discount_pct/100))*o.quantity),1))desc)  
as Ranking from customers as c inner join orders as o  
on c.customer_id=o.customer_id group by c.Customer_id ;
```

OUTPUT



The screenshot shows a 'Result Grid' window with a table of customer data. The table has three columns: 'Customer_id', 'Spend', and 'Ranking'. The data is sorted by 'Spend' in descending order, with the highest spender (Customer_id 2) at the top, ranked 1. The grid includes a 'Filter Rows' button and a scroll bar on the left.

Customer_id	Spend	Ranking
2	35008.8	1
5	28887.6	2
11	18198.9	3
12	15672.2	4
15	15133.4	5
7	12949.6	6
4	11957.7	7
23	11413.9	8
22	8897.1	9
20	8792.6	10
1	8657.4	11
21	8317.2	12

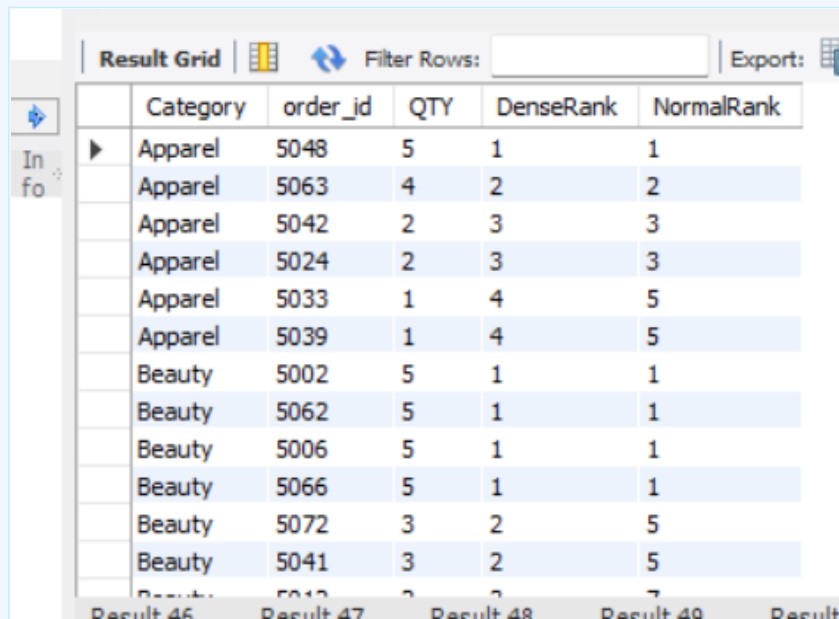
Q51

For every category, rank the orders by quantity, from the highest quantity to the lowest. Show this two ways side by side: one where tied orders get the same rank but the rank number skips ahead afterward, and one where tied orders get the same rank with no numbers skipped.

SQL QUERY

```
select Category,order_id ,Quantity as QTY, dense_rank() over (partition by Category order by
    Quantity desc) as DenseRank,
rank() over (partition by category order by Quantity desc) as NormalRank
from orders;
```

OUTPUT



Category	order_id	QTY	DenseRank	NormalRank
Apparel	5048	5	1	1
Apparel	5063	4	2	2
Apparel	5042	2	3	3
Apparel	5024	2	3	3
Apparel	5033	1	4	5
Apparel	5039	1	4	5
Beauty	5002	5	1	1
Beauty	5062	5	1	1
Beauty	5006	5	1	1
Beauty	5066	5	1	1
Beauty	5072	3	2	5
Beauty	5041	3	2	5

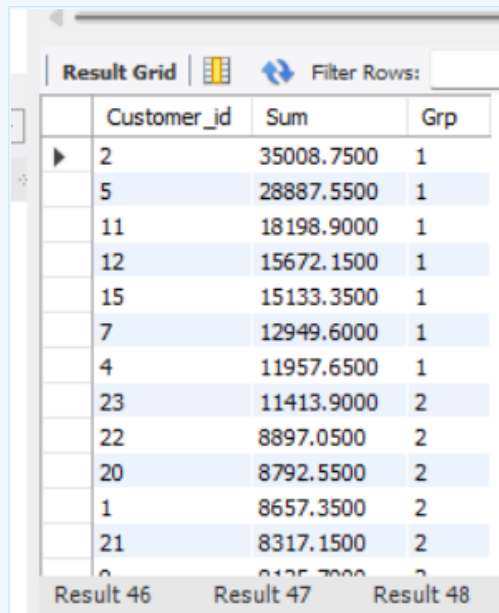
Q52

Divide the customers into 4 equal groups based on their total spend, where group 1 is the highest spending group.

SQL QUERY

```
select Customer_id,sum((unit_price-unit_price*(discount_pct/100))*quantity) as Sum ,ntile(4) over
(order by sum((unit_price-unit_price*(discount_pct/100))*quantity)desc ) as Grp from orders group by
Customer_id;
```

OUTPUT



The screenshot shows a 'Result Grid' window with a table containing customer IDs, their total spend (Sum), and their assigned group (Grp). The data is sorted by total spend in descending order. The first 7 rows belong to Group 1, the next 5 rows to Group 2, and the last 3 rows to Group 3. The table has 15 rows in total, with the last three rows (IDs 2, 3, and 4) partially cut off at the bottom of the grid.

Customer_id	Sum	Grp
2	35008.7500	1
5	28887.5500	1
11	18198.9000	1
12	15672.1500	1
15	15133.3500	1
7	12949.6000	1
4	11957.6500	1
23	11413.9000	2
22	8897.0500	2
20	8792.5500	2
1	8657.3500	2
21	8317.1500	2
3	8125.7000	3
4	7125.7000	3

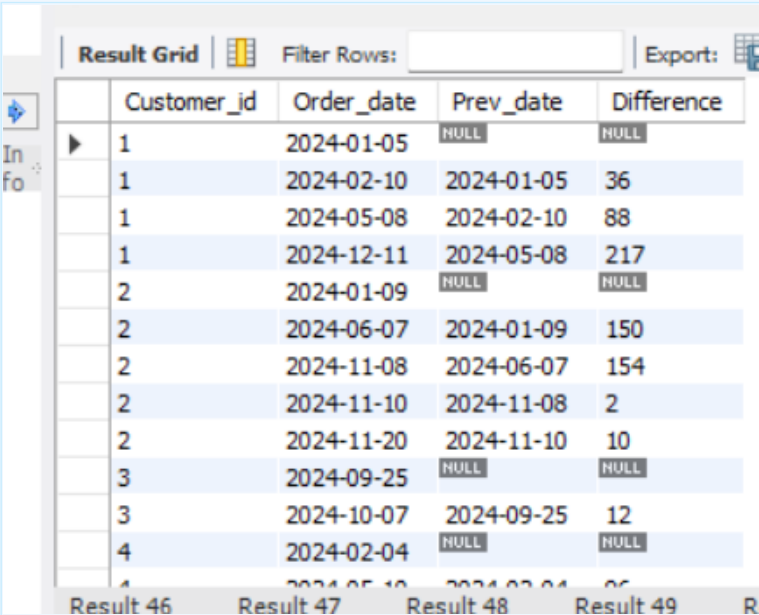
Q53

For each customer, show the previous order date and the gap in days between two consecutive orders.

SQL QUERY

```
with cte as
(
select Customer_id,Order_date,lag(order_date) over ( partition by Customer_id order by Order_date)
as Prev_date from orders
)
select Customer_id, Order_date,Prev_date,datediff(order_date,prev_date) as Difference from cte;
```

OUTPUT



The screenshot shows a SQL query result grid with the following columns: Customer_id, Order_date, Prev_date, and Difference. The data is sorted by Customer_id and then by Order_date. The 'Prev_date' column shows the previous order date for each customer, and the 'Difference' column shows the number of days between consecutive orders. The first order for each customer has a NULL value for Prev_date and Difference.

	Customer_id	Order_date	Prev_date	Difference
▶	1	2024-01-05	NULL	NULL
	1	2024-02-10	2024-01-05	36
	1	2024-05-08	2024-02-10	88
	1	2024-12-11	2024-05-08	217
	2	2024-01-09	NULL	NULL
	2	2024-06-07	2024-01-09	150
	2	2024-11-08	2024-06-07	154
	2	2024-11-10	2024-11-08	2
	2	2024-11-20	2024-11-10	10
	3	2024-09-25	NULL	NULL
	3	2024-10-07	2024-09-25	12
	4	2024-02-04	NULL	NULL
	4	2024-05-10	2024-02-04	86

Result 46 Result 47 Result 48 Result 49 R

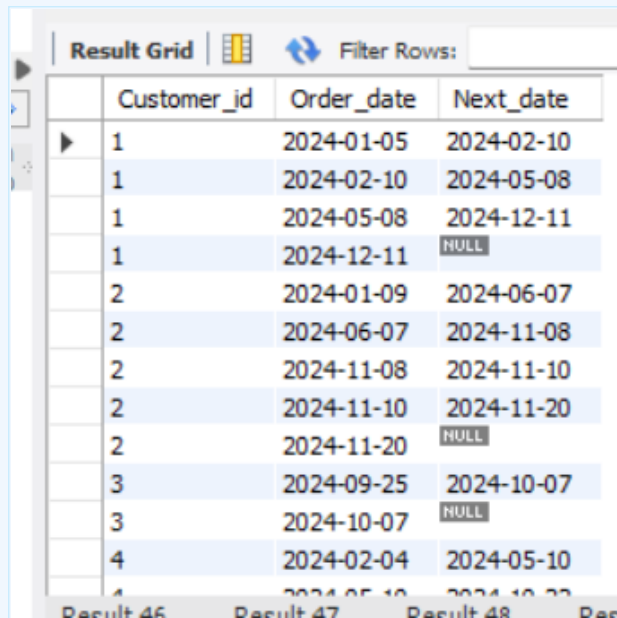
Q54

For each customer, show the next order date after the current one.

SQL QUERY

```
select Customer_id, Order_date, lead(order_date) over ( partition by Customer_id order by Order_date)
as Next_date from orders;
```

OUTPUT



The screenshot shows a 'Result Grid' window with a 'Filter Rows' input. The grid displays the results of the SQL query, showing columns for Customer_id, Order_date, and Next_date. The data is grouped by Customer_id, and the Next_date is the next order date for each customer, or NULL if there are no more orders.

	Customer_id	Order_date	Next_date
▶	1	2024-01-05	2024-02-10
	1	2024-02-10	2024-05-08
	1	2024-05-08	2024-12-11
	1	2024-12-11	NULL
	2	2024-01-09	2024-06-07
	2	2024-06-07	2024-11-08
	2	2024-11-08	2024-11-10
	2	2024-11-10	2024-11-20
	2	2024-11-20	NULL
	3	2024-09-25	2024-10-07
	3	2024-10-07	NULL
	4	2024-02-04	2024-05-10
	4	2024-05-10	2024-10-22

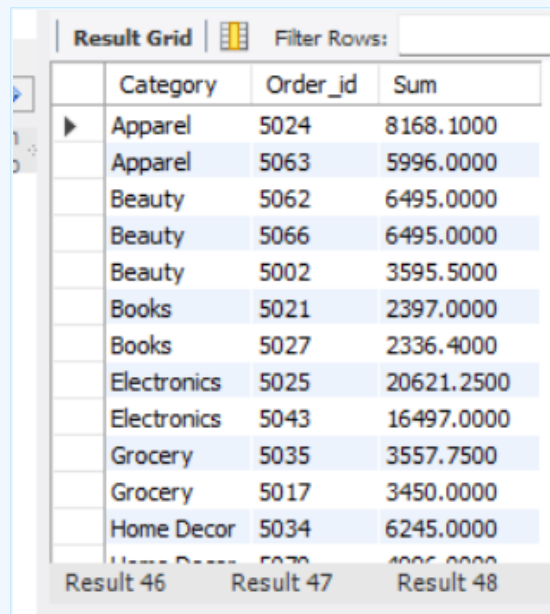
Q55

Show the top 2 orders by net amount in each category.

SQL QUERY

```
with cte as
(
select Category,order_id,(unit_price-unit_price*(discount_pct/100))*quantity as Sum ,
dense_rank() over ( partition by category order by
(unit_price-unit_price*(discount_pct/100))*quantity desc) as Ranking
from orders
)
Select Category,Order_id,Sum from cte
where Ranking<=2;
```

OUTPUT

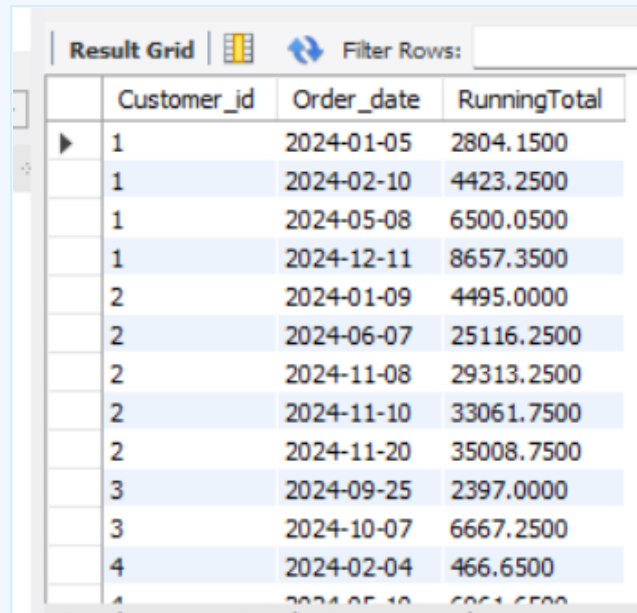


The screenshot shows a 'Result Grid' with columns: Category, Order_id, and Sum. The data is sorted by Category and then by Sum in descending order. The top 2 orders for each category are highlighted. The categories and their top 2 orders are: Apparel (5024, 5063), Beauty (5062, 5066), Books (5021, 5027), Electronics (5025, 5043), Grocery (5035, 5017), and Home Decor (5034, 5030). The bottom of the grid shows 'Result 46', 'Result 47', and 'Result 48'.

Category	Order_id	Sum
Apparel	5024	8168.1000
Apparel	5063	5996.0000
Beauty	5062	6495.0000
Beauty	5066	6495.0000
Beauty	5002	3595.5000
Books	5021	2397.0000
Books	5027	2336.4000
Electronics	5025	20621.2500
Electronics	5043	16497.0000
Grocery	5035	3557.7500
Grocery	5017	3450.0000
Home Decor	5034	6245.0000
Home Decor	5030	4006.0000
Result 46	Result 47	Result 48

Q56**For each customer, show a running total of their spend in order of date.****SQL QUERY**

```
select Customer_id,Order_date,sum((unit_price-unit_price*(discount_pct/100))*quantity)
over (partition by customer_id order by order_date) as RunningTotal from orders order by
Customer_id,order_date;
```

OUTPUT

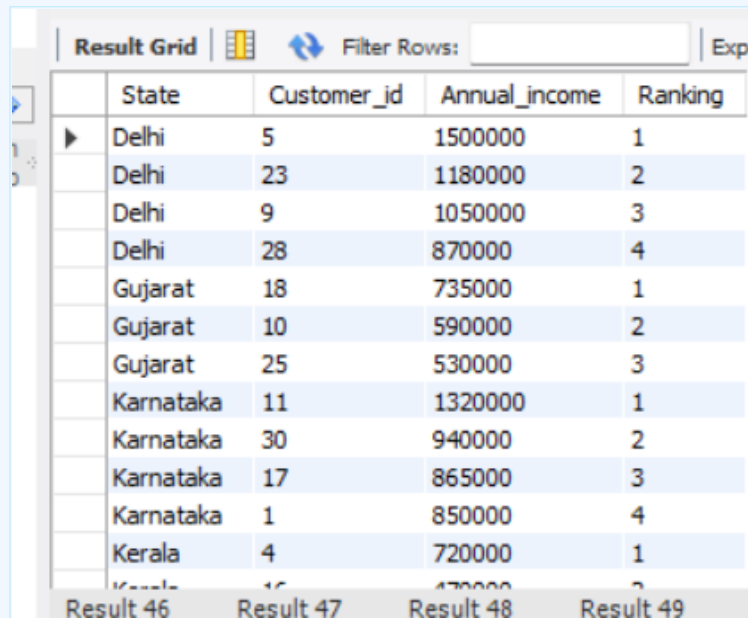
	Customer_id	Order_date	RunningTotal
▶	1	2024-01-05	2804.1500
	1	2024-02-10	4423.2500
	1	2024-05-08	6500.0500
	1	2024-12-11	8657.3500
	2	2024-01-09	4495.0000
	2	2024-06-07	25116.2500
	2	2024-11-08	29313.2500
	2	2024-11-10	33061.7500
	2	2024-11-20	35008.7500
	3	2024-09-25	2397.0000
	3	2024-10-07	6667.2500
	4	2024-02-04	466.6500
	4	2024-05-10	6061.6500

Q57

Within each state, order the customers from the highest annual income to the lowest and number them starting from 1 in every state.

SQL QUERY

```
select State, Customer_id, Annual_income,
       row_number() over (partition by state order by Annual_income desc) as Ranking
from customers;
```

OUTPUT

The screenshot shows a 'Result Grid' window with a table of customer data. The table has four columns: State, Customer_id, Annual_income, and Ranking. The data is grouped by state, with each state's customers ordered by their annual income in descending order. The ranking starts at 1 for the highest income customer in each state.

	State	Customer_id	Annual_income	Ranking
▶	Delhi	5	1500000	1
	Delhi	23	1180000	2
	Delhi	9	1050000	3
	Delhi	28	870000	4
	Gujarat	18	735000	1
	Gujarat	10	590000	2
	Gujarat	25	530000	3
	Karnataka	11	1320000	1
	Karnataka	30	940000	2
	Karnataka	17	865000	3
	Karnataka	1	850000	4
	Kerala	4	720000	1
	Kerala	16	470000	2

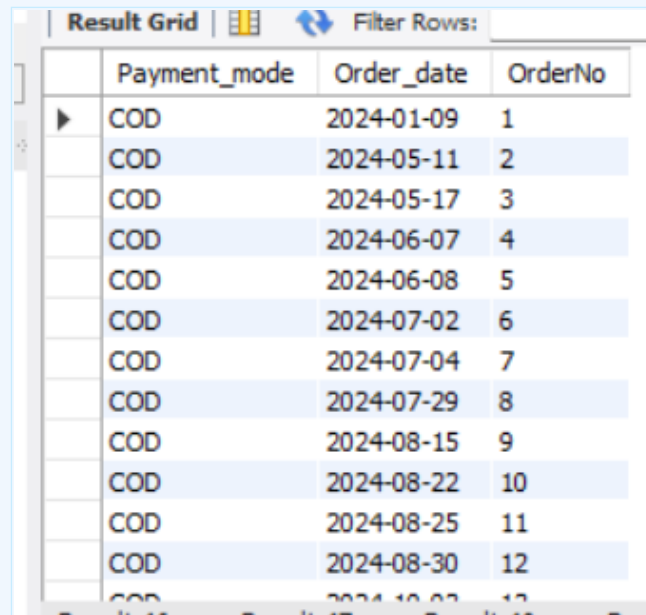
Result 46 Result 47 Result 48 Result 49

Q58

For each payment mode, number the orders from the earliest order date to the most recent.

SQL QUERY

```
select Payment_mode,Order_date,row_number() over (partition by payment_mode order by
Order_date,order_id) as OrderNo from orders;
```

OUTPUT

	Payment_mode	Order_date	OrderNo
▶	COD	2024-01-09	1
	COD	2024-05-11	2
	COD	2024-05-17	3
	COD	2024-06-07	4
	COD	2024-06-08	5
	COD	2024-07-02	6
	COD	2024-07-04	7
	COD	2024-07-29	8
	COD	2024-08-15	9
	COD	2024-08-22	10
	COD	2024-08-25	11
	COD	2024-08-30	12

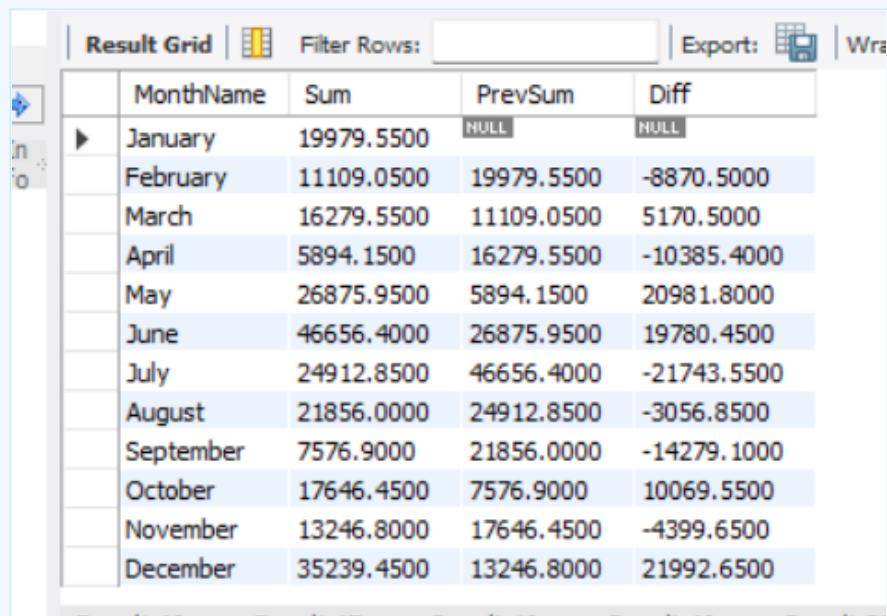
Q59

Show month wise revenue along with the previous month revenue and the change between them.

SQL QUERY

```
with cte as (  
  select month(order_date) as MonthNo, monthname(order_date) as  
    MonthName, sum((unit_price-unit_price*(discount_pct/100))*quantity) as Sum  
  , lag(sum((unit_price-unit_price*(discount_pct/100))*quantity)) over(order by month(order_date)) as  
    PrevSum  
  from orders group by month(order_date), monthname(order_date) order by month(order_date)  
)  
select MonthName, Sum, PrevSum, Sum-PrevSum as Diff from cte;
```

OUTPUT



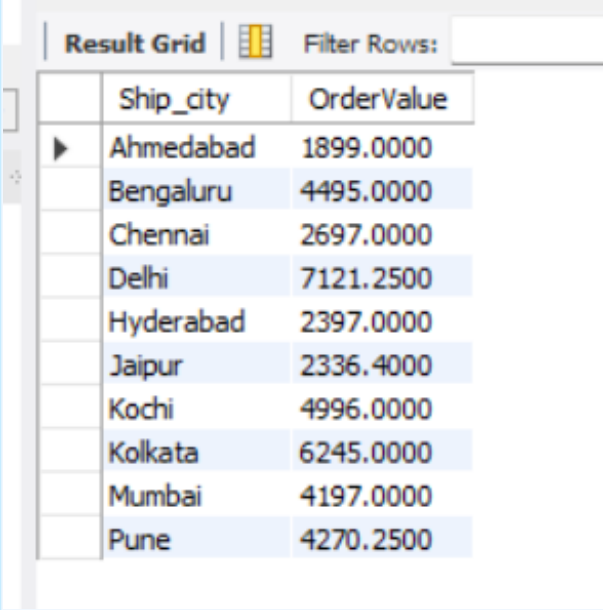
MonthName	Sum	PrevSum	Diff
January	19979.5500	NULL	NULL
February	11109.0500	19979.5500	-8870.5000
March	16279.5500	11109.0500	5170.5000
April	5894.1500	16279.5500	-10385.4000
May	26875.9500	5894.1500	20981.8000
June	46656.4000	26875.9500	19780.4500
July	24912.8500	46656.4000	-21743.5500
August	21856.0000	24912.8500	-3056.8500
September	7576.9000	21856.0000	-14279.1000
October	17646.4500	7576.9000	10069.5500
November	13246.8000	17646.4500	-4399.6500
December	35239.4500	13246.8000	21992.6500

Q60**Find the second highest order value in each ship city.**

SQL QUERY

```
with cte as
(
select Ship_City,(unit_price-unit_price*(discount_pct/100))*quantity as Ordervalue,dense_rank() over
(partition by Ship_city order by (unit_price-unit_price*(discount_pct/100))*quantity desc) as
    OrderValueRank
from orders
)
select Ship_city,OrderValue from cte where OrderValueRank=2;
```

OUTPUT



Ship_city	OrderValue
Ahmedabad	1899.0000
Bengaluru	4495.0000
Chennai	2697.0000
Delhi	7121.2500
Hyderabad	2397.0000
Jaipur	2336.4000
Kochi	4996.0000
Kolkata	6245.0000
Mumbai	4197.0000
Pune	4270.2500

Q61

For every customer who has ordered, show their first order date, last order date and the number of days between the two.

SQL QUERY

```
select c.Customer_id , min(o.order_date) FirstDate,max(o.order_date) as LastDate ,
       datediff(max(o.order_date),min(o.order_date))
as DaysBetween from customers as c inner join orders as o on c.customer_id=o.customer_id group by
c.Customer_id;
```

OUTPUT

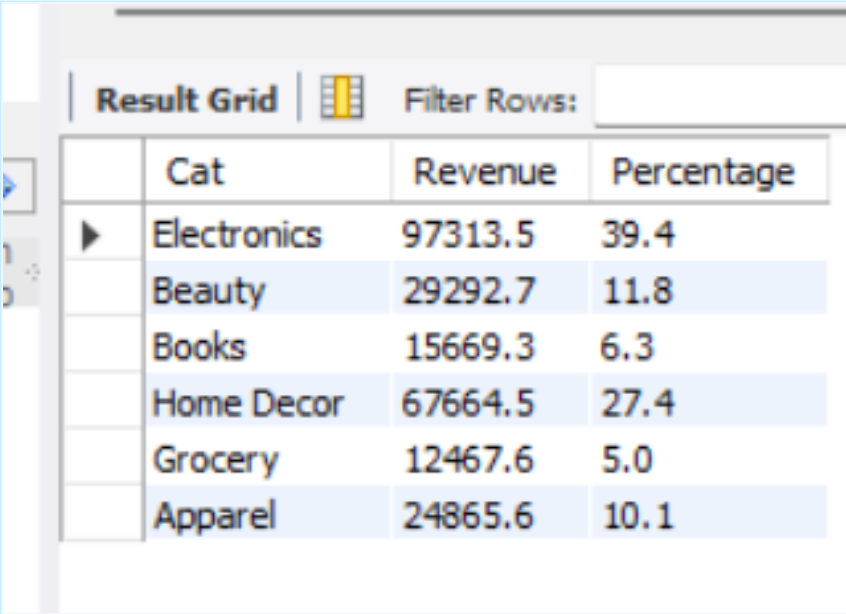
	Customer_id	FirstDate	LastDate	DaysBetween
▶	7	2024-02-12	2024-10-19	250
	23	2024-03-02	2024-12-16	289
	2	2024-01-09	2024-11-20	316
	3	2024-09-25	2024-10-07	12
	9	2024-06-10	2024-12-08	181
	5	2024-03-25	2024-11-13	233
	25	2024-01-08	2024-01-08	0
	1	2024-01-05	2024-12-11	341
	6	2024-02-29	2024-02-29	0
	20	2024-04-28	2024-08-15	109
	24	2024-01-12	2024-08-18	219
	4	2024-02-04	2024-10-22	261
	10	2024-02-05	2024-05-25	110

Q62**Show each category's revenue and what percentage it contributes to the total revenue.**

SQL QUERY

```
with cte as (  
  select Category as Cat,  
         sum((unit_price-unit_price*(discount_pct/100))*quantity) as Revenue  
  from orders  
 group by Category  
)  
select Cat, round(Revenue,1) as Revenue, round(Revenue*100/(select sum(Revenue) from cte),1) as  
       Percentage from cte;
```

OUTPUT



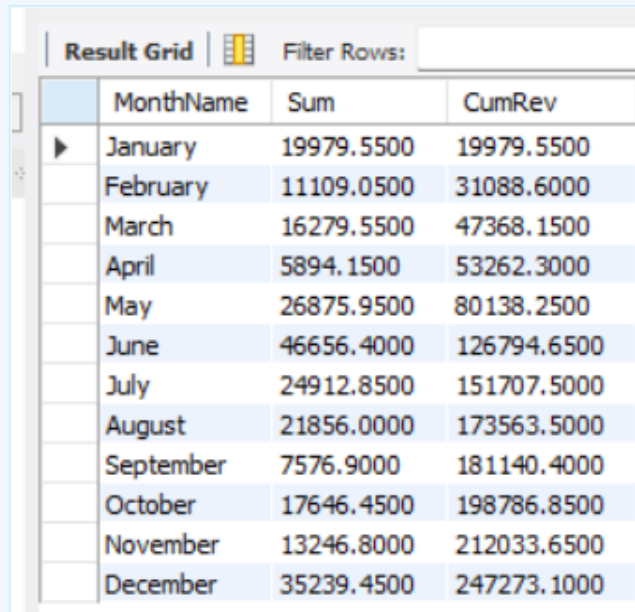
	Cat	Revenue	Percentage
▶	Electronics	97313.5	39.4
	Beauty	29292.7	11.8
	Books	15669.3	6.3
	Home Decor	67664.5	27.4
	Grocery	12467.6	5.0
	Apparel	24865.6	10.1

Q63**Show month wise revenue along with the cumulative revenue for the year.**

SQL QUERY

```
with cte as (  
  select month(order_date) as MonthNo, monthname(order_date) as  
    MonthName, sum((unit_price-unit_price*(discount_pct/100))*quantity) as Sum  
  from orders group by month(order_date), monthname(order_date) order by month(order_date)  
)  
select MonthName, Sum, Sum(sum) over (order by MonthNo) as CumRev from cte;
```

OUTPUT



The screenshot shows a 'Result Grid' window with a 'Filter Rows' field. The grid contains a table with four columns: 'MonthName', 'Sum', and 'CumRev'. The data is organized by month from January to December. The 'Sum' column represents the monthly revenue, and the 'CumRev' column represents the cumulative revenue up to that month.

MonthName	Sum	CumRev
January	19979.5500	19979.5500
February	11109.0500	31088.6000
March	16279.5500	47368.1500
April	5894.1500	53262.3000
May	26875.9500	80138.2500
June	46656.4000	126794.6500
July	24912.8500	151707.5000
August	21856.0000	173563.5000
September	7576.9000	181140.4000
October	17646.4500	198786.8500
November	13246.8000	212033.6500
December	35239.4500	247273.1000

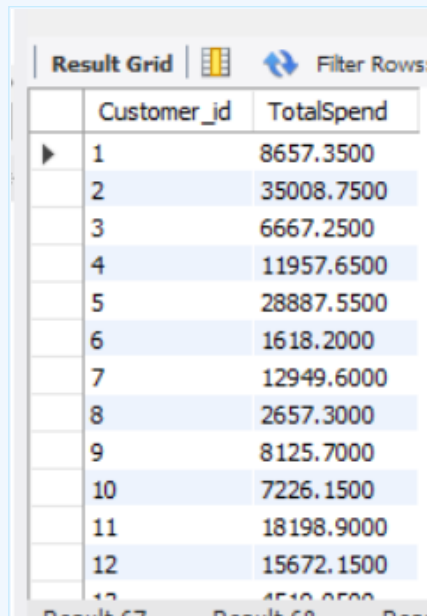
Q64

List every customer, whether they ordered or not, along with their total spend, showing 0 for customers with no orders.

SQL QUERY

```
select c.Customer_id,coalesce(sum((o.unit_price-o.unit_price*(o.discount_pct/100))*o.quantity),0) as  
    TotalSpend  
from customers as c left join orders as o on c.customer_id=o.customer_id group by c.Customer_id;
```

OUTPUT



The screenshot shows a 'Result Grid' window with a 'Filter Rows' button. It displays a table with two columns: 'Customer_id' and 'TotalSpend'. The table contains 12 rows of data, with the first row highlighted. The data is as follows:

Customer_id	TotalSpend
1	8657.3500
2	35008.7500
3	6667.2500
4	11957.6500
5	28887.5500
6	1618.2000
7	12949.6000
8	2657.3000
9	8125.7000
10	7226.1500
11	18198.9000
12	15672.1500

Q65

For every order, compare its net amount to the average net amount of its own category, and label it 'Above Average' or 'Below Average' accordingly.

SQL QUERY

```
select Order_id,Category,(unit_price-unit_price*(discount_pct/100))*quantity as Amount,
(avg((unit_price-unit_price*(discount_pct/100))*quantity) over (partition by category)) as
    Category_Average,
case
when
    (unit_price-unit_price*(discount_pct/100))*quantity>(avg((unit_price-unit_price*(discount_pct/100))*quantity) over (partition by category)) then "Above Average"
when
    (unit_price-unit_price*(discount_pct/100))*quantity<(avg((unit_price-unit_price*(discount_pct/100))*quantity) over (partition by category)) then "Below Average"
else "Average"
end as Type
from orders;
```

OUTPUT

Order_id	Category	Amount	Category_Average	Type
5042	Apparel	3228.3000	4144.26666667	Below Average
5063	Apparel	5996.0000	4144.26666667	Above Average
5039	Apparel	2804.1500	4144.26666667	Below Average
5048	Apparel	3245.0000	4144.26666667	Below Average
5024	Apparel	8168.1000	4144.26666667	Above Average
5033	Apparel	1424.0500	4144.26666667	Below Average
5044	Beauty	1278.4000	2662.97272727	Below Average
5002	Beauty	3595.5000	2662.97272727	Above Average
5072	Beauty	2397.0000	2662.97272727	Below Average
5066	Beauty	6495.0000	2662.97272727	Above Average
5006	Beauty	1745.0000	2662.97272727	Below Average
5012	Beauty	593.3000	2662.97272727	Below Average